计算机组成原理与系统结构



第6章 指令系统

杭州電子科技大學
HANGZHOU DIANZI UNIVERSITY

第6章 指令系统

- 6.1 指令系统概述
- 6.2 指令格式
- 6.3 寻址方式
- 6.4 指令系统的设计
- 6.5 指令系统举例
- 6.6 RISC-V指令系统
- 6.7 ARM指令系统
- 本章小结

6.1 指令系统概述



机器指令与指令系统

指令信息

指令集体系架构

6.1.1 机器指令与指令系统

- ①计算机指令：指示计算机硬件进行某种操作的命令。
- ②机器指令：使用二进制编码来表示的计算机指令
  - 能被计算机硬件识别并执行的特殊的0/1代码串（又称机器字）
  - 指定了计算机硬件的某种特定操作，计算机硬件结构密切相关
  - 不同的计算机，硬件能够识别并执行的机器指令也不相同
- ③机器语言：使用机器指令来编写程序的规范，是计算机硬件的语言系统
  - 面向机器的语言
- ④机器语言程序：机器指令的有序集合
  - 用机器指令编写的二进制程序（又称机器码）
  - 可以直接被计算机硬件识别并执行

6.1.1 机器指令与指令系统

- ⑤指令系统（Instruction Set）：指一台计算机的所有机器指令的集合
  - 指令集，它体现了计算机硬件的功能或者所能执行的全部操作
- 指令系统在计算机中的地位：软件和硬件的交界面和接口
- ⑥汇编指令：机器指令的二进制代码→助记符
  - 汇编语言：用汇编指令书写程序的表达方式
    - 面向机器的语言，和机器指令几乎一一对应
  - 汇编指令集：计算机所有汇编指令的集合
  - 汇编语言程序：汇编指令编写的程序
  - 汇编器：将汇编语言程序 翻译成 机器语言程序 的程序



6.1.2 指令信息

- 指令信息：4个
  - 指令的功能：要执行的操作，一般有运算、传送、移位、跳转等
  - 操作的对象：要处理或参加运算的数据，称为源操作数（Source Operand）
  - 操作的结果：存放运算或处理结果的数据，称为目的操作数（Destination Operand）
  - 下一条指令地址：即指出下一条指令在主存的地址

- 不同的CPU有不同的指令系统
- 不同之处：指令格式、指令类型、寻址方式、指令条数



6.1.3 指令集体系架构

- **指令集体系结构ISA：Instruction-Set Architecture**
  - 处理器所支持的指令集（汇编指令集及机器指令编码）及其操作数的相关信息（寄存器、存储器、I/O等）
- **ISA的作用：在系统软件设计者和处理器硬件设计者之间提供了一个抽象层**
  - 软件设计者依据ISA知道如何编写机器语言程序来控制与使用处理器硬件
  - 硬件设计者依据ISA来设计微体系结构，并实现能执行机器语言的处理器硬件
- **ISA的内容：**
  - ①核心：可执行的指令集合，包括指令格式、指令类型、指令功能、指令编码等
  - ②指令可以接受的数据类型
  - ③操作数可以存放的寄存器组和结构
  - ④操作数所能存放的存储空间（存储器和I/O）大小、编址方式、大小端模式等
  - ⑤指令获取操作数的方式，即数据寻址方式
  - ⑥指令执行的控制方式，即指令寻址方式，包括程序计数器、条件码定义等

6.1.3 指令集体系架构 Intel x86 IA-32的ISA特征

ISA特征	x86架构IA-32的特征简述
指令集	CISC指令集，变长指令字（1B~13B）、变长操作码（1B~2B），指令集复杂，指令条数：92-300+
指令格式	包含操作码字段OP、寻址方式码字段Mod/RM、基址变址字段SIB、偏移量/地址/立即数字段等
寻址方式	数据寻址有寄存器寻址、立即寻址、直接寻址、寄存器间接寻址、变址寻址（寄存相对寻址）、基址变址寻址、基址变址+偏移量寻址、比例变址寻址等8种寻址方式；指令顺序寻址由CS：EIP指定；跳跃寻址有直接、相对、间接三种方式，用于转移和调用指令；条件转移的条件码由Eflags规定
数据类型	有符号和无符号整数，可按字节（8位）、字（16位）、双字（32位）访问，有符号数用补码表示；BCD数据；字符串数据；实数支持IEEE754单精度和双精度浮点数
寄存器	8个32位通用寄存器（EAX、EBX、ECX、EDX、ESP、EBP、ESI、EDI），可按8位、16位和32位访问；32位标志寄存器Eflags（含条件码和控制位）；32位指令指针EIP；6个16位的段寄存器CS、DS、ES、SS、FS、GS
存储器	地址32/36位，按字节编址，寻址空间4GB/64GB；多字节存储顺序是小端模式
工作模式	有实地址模式、保护模式、虚拟8086模式、系统管理模式四种
I/O编址方式	I/O独立编址，地址16位，寻址空间64KB；支持硬中断和软中断两种，256个中断类型号与中断向量

6.2 指令格式

- 6.2.1 指令格式与字长
- 6.2.2 地址码
- 6.2.3 操作数类型
- 6.2.4 指令操作码及扩展技术

6.2.1 指令格式与字长

❖ 1、指令基本格式：由操作码和地址码两部分组成

操作码字段（OP）	地址码字段（A）

❖ 指令信息：指令的功能、操作的对象、操作的结果、下一条指令地址

❖ 操作码：指明该指令所要完成的操作，也规定了指令格式

- 操作码的编码：必须是唯一的
- 位数：反映了计算机的操作种类，即计算机允许的指令条数；
- n位二进制操作码：最多可表示2^n种指令。

❖ 地址码：寻找 操作数 或者 下一条指令地址

- 操作数地址 or 下一条指令的地址

6.2.1 指令格式与字长

❖ 2、指令字长

- ①指令字：机器指令的二进制编码，也称为机器字
- ②指令字长：一条机器指令中所包含的二进制码的位数
  - 不同计算机的指令字长可能不同，主要取决于操作码长度、地址码长度和地址码个数
- ③机器字长：即计算机字长，指计算机能一次处理的二进制数据的位数
  - 体现了计算机的运算能力和精度；通常是字节的整数倍，8、16、32或64位

❖ 按照指令字长与机器字长的关系分类：

- ④短格式指令：指令字长≤机器字长
- ⑤长格式指令：指令字长 > 机器字长
- 短字长指令所占空间少，长字长指令可表示更多操作信息；可以并存于同一个指令系统；
- 最常用的指令：设计成短格式指令，可以节省存储空间、提高指令的执行速度

6.2.1 指令格式与字长

❖ 2、指令字长

❖ 指令长度是否固定?

- ⑥固定指令字长的指令：所有指令的字长均相等，一般等于机器字长。
  - 多用于RISC指令系统：32位的MIPS、ARM、SPARC处理器的指令字长固定为32位
- ⑦可变指令字长的指令：指令字长不固定，通常取字节的整数倍
  - 单字长指令：指令字长=机器字长
  - 双字长指令：指令字长=机器字长*2
  - 半字长、三字长等
  - 将操作码放在第一次读取的字节内，以便知道这是一条什么指令，该指令有多长，还应该取几次指令码
  - IBM 370：指令字长可变，16位（半个字）、32位（一个字）、48位（一字半）
  - Intel 8086：指令字长有6种，8、16、24、32、40和48位



6.2.2 地址码

- 地址码作用：用来寻找执行指令所需要的操作数或者下条指令地址。
- 操作数：分为源操作数和目的操作数。
- 地址码的含义：取决于寻址方式
  - 可以是：操作数本身 / 操作数的地址 / 下一条指令的地址
  - 可以是：主存的地址 / 寄存器的编号 / I/O设备的端口地址
- 根据地址数，可分为四种指令格式

6.2.2 地址码



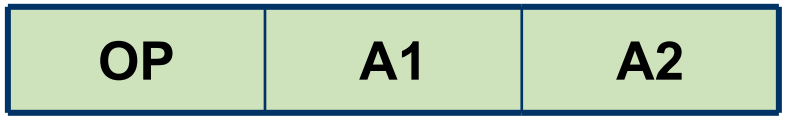
（1）三地址指令：

- （A1）OP（A2）→A3



（3）单地址指令：

- （ACC）OP（A）→ACC
  - 一个操作数隐含规定
- OP（A）→A
  - 单目操作：如NEG、INC等



（2）二地址指令：

- （A1）OP（A2）→A1
- A1：目的操作数
- A2：源操作数



（4）零地址指令

- 不涉及操作数：如NOP、HLT指令
- 操作数隐含：如PUSH、POP指令

❖ 各种地址格式的指令，可以并存于一个指令系统中。

6.2.3 操作数类型

- 按照操作数存放位置分：
- 立即数类型：操作数是常数，存放在指令地址字段A中
  - 地址码字段A：立即数本身，通常标记为imm或者Data
- 寄存器类型：操作数存放在CPU的通用寄存器中，A为寄存器编号/地址
  - MIPS、RISC-V：32个寄存器，寄存器编号/地址为5位二进制
  - n是寄存器序号，对应寄存器记为R_n；操作数记为Reg[R_n]
- 存储器类型：操作数存放在主存中，A为其地址信息
  - A可以是：直接地址、间接地址、形式地址、相对偏移量等，取决于该操作数的寻址方式
  - 有效地址EA（Effective Address）：根据寻址方式，计算出操作数存放在存储器的单元地址（对应于虚拟存储器的线性地址）
  - 操作数：标记为Mem[EA]
- I/O数据类型：操作数存放在I/O设备接口中，A为其端口地址Port
  - 操作数标记为IO[Port]

6.2.3 操作数类型

❖数据扩展：地址码A较短，扩展变长后再参与计算或者寻址，两种扩展方法

- 符号扩展（signed extended）：将最高位当作符号位，填充到所有的高位，
  - 一般用于扩展带符号立即数操作数、相对偏移量等
- 零扩展（unsigned extended）：用0填充到所有的高位
  - 一般用于扩展无符号立即数操作数、地址信息等。

❖例1：将7FH符号扩展成16位?

❖007FH

❖例2：将80H符号扩展成16位?

❖FF80H

6.2.3 操作数类型

- 按照操作数物理含义（数据类型）性质分：
- ①数值数据：整数、浮点数、十进制数，带符号数 / 无符号数
  - 主要操作：各种运算、移位、传送等
- ②指针或地址：指存储器地址，由访问存储器的指令计算出，它是无符号整数。
  - 主要操作：用于读写存储器、计算与获取有效地址
- ③位或位串数据：一般用于表示标志位、状态位、控制位等信息
  - 主要操作：置位、复位、测试等
- ④字符或者字符串：一般用于表示文本信息，包括西文字符(ASCII码)和中文字(内码)等
  - 主要操作：移动、复制或者其他处理
- ⑤逻辑数据：用0/1表示真/假的逻辑值，通常用于条件转移指令中对条件码的检测。
- 若只给出操作数的二进制代码，不加任何说明，我们无法知道它代表什么，关键是要看程序或者指令如何对它进行处理。



6.2.4 指令操作码及扩展技术



1、指令操作码

2、操作码扩展技术

1、指令操作码

- 操作码的长度：决定了指令的条数
- 操作码的编码：定长 / 可变
- ①定长编码：编码长度固定，集中放在一个或几个字段内
  - 便于硬件设计，指令译码时间短，广泛用于大中型、超级小型计算机以及RISC中
  - IBM370和VAX-11系列机，操作码长度均为8位，能容纳2^8=256条指令
- ②变长编码：长度不固定，不同指令的操作码长短不一，分散在不同字段
  - 增加了指令译码难度，但是可有效地压缩操作码的平均长度，被广泛采用
  - 设计原则：安排指令使用频度高的指令占用短的操作码，对使用频度低的指令可占用较长的操作码
  - 达到效果：可以压缩程序存储空间、缩短指令的译码时间
  - PDP-11的操作码4-16位不等，Intel 8086/80386的操作码6-16位不等

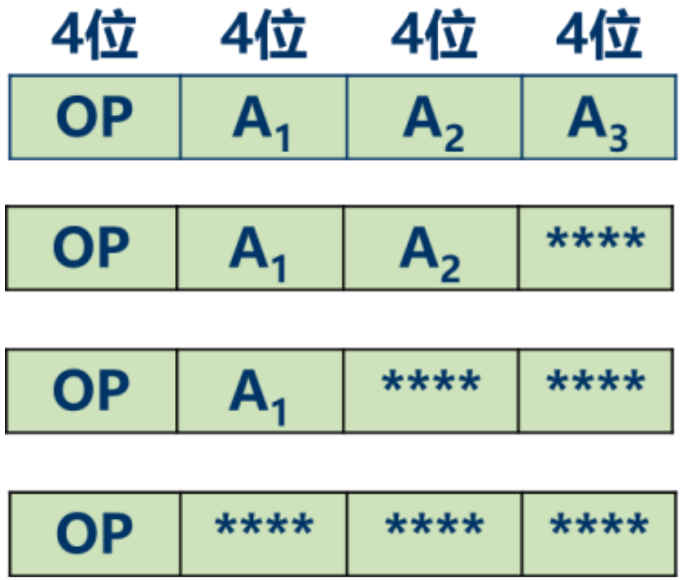
2、操作码扩展技术

- **操作码扩展技术：一种重要的指令优化技术**
  - 将不使用的地址码字段扩展为操作码。
  - 操作码的长度随地址数的减少而增加，即不同地址数的指令可以具有不同长度的操作码
  - 优点：缩短指令的平均长度，增加指令字所能表示的操作信息。
  - 缺点：需要更多的硬件支持，它的指令译码更加复杂，使控制器设计难度增大。

2、操作码扩展技术

- 【例6.1】假设一个指令系统，指令字长16位，每个地址码长度4位；指令系统中，含有三地址指令、二地址指令、单地址指令和零地址指令各若干条。
  - （1）如果采用定长编码的操作码格式，请问，该指令系统最多共有多少条指令?

- 三地址指令的地址最多，每个地址4位，3个地址12位；则OP的位数=16-12=4位，故指令条数最多2^4=16条



2、操作码扩展技术

容纳了61条指令

- 【例6.1】
  - （2）如果采用操作码扩展技术，请设计一个指令系统操作码编码方案，尽量多地、平衡地容纳各种地址格式的指令；并计算设计方案中，指令系统能容纳多少条指令？

OP	A_1	A_2	A_3
0000 0001 : 1110	A_1 A_1 : A_1	A_2 A_2 : A_2	A_3 A_3 : A_3
1111 1111 : 1111	0000 0001 : 1110	A_2 A_2 : A_2	A_3 A_3 : A_3
1111 1111 : 1111	1111 1111 : 1111	0000 0001 : 1110	A_3 A_3 : A_3
1111 1111 : 1111	1111 1111 : 1111	1111 1111 : 1111	0000 0001 : 1111

4位操作码,15条三地址指令

8位操作码,15条二地址指令

12位操作码,15条一地址指令

16位操作码，16条零地址指令

2、操作码扩展技术

容纳了61条指令

- 【例6.1】
  - （3）尝试设计一个指令译码器，实现上述变长编码的操作码译码。

4位操作码,15条三地址指令

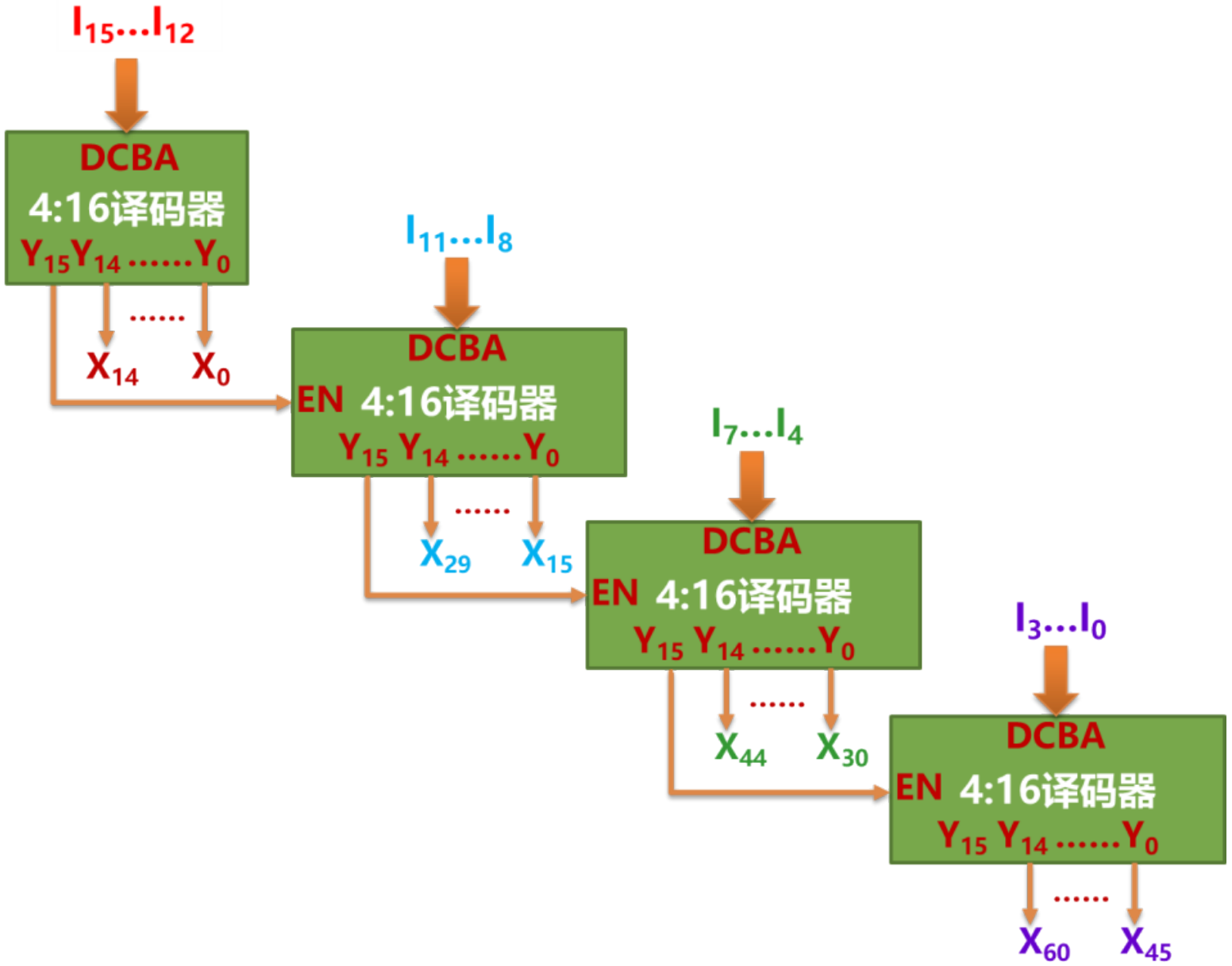
8位操作码,15条二地址指令

12位操作码,15条一地址指令

16位操作码，16条零地址指令

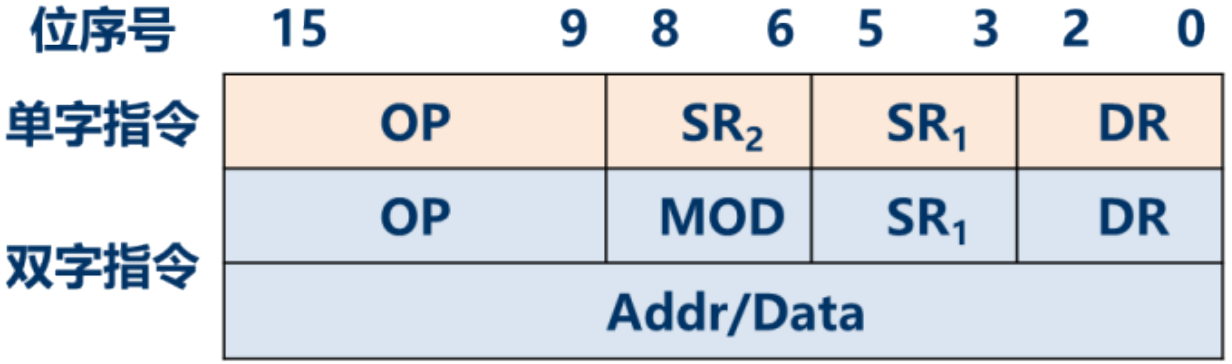
	$I_{15}...I_{12}$	$I_{11}...I_8$	$I_7...I_4$	$I_3...I_0$
	OP	A_1	A_2	A_3
X_0	0000	A_1	A_2	A_3
X_1	0001	A_1	A_2	A_3
:	:	:	:	:
X_{14}	1110	A_1	A_2	A_3
X_{15}	1111	0000	A_2	A_3
X_{16}	1111	0001	A_2	A_3
:	:	:	:	:
X_{29}	1111	1110	A_2	A_3
X_{30}	1111	1111	0000	A_3
X_{31}	1111	1111	0001	A_3
:	:	:	:	:
X_{44}	1111	1111	1110	A_3
X_{45}	1111	1111	1111	0000
X_{46}	1111	1111	1111	0001
:	:	:	:	:
X_{60}	1111	1111	1111	1111

2、操作码扩展技术


I15…I12
DCBA
4:16译码器
Y15Y14 ……Y0
……
X14
X0
I11…I8
DCBA
EN 4:16译码器
Y15 Y14 ……Y0
……
X29
X15
I7…I4
DCBA
EN 4:16译码器
Y15 Y14 ……Y0
……
X44
X30
I3…I0
DCBA
EN 4:16译码器
Y15 Y14 ……Y0
……
X60
X45

2、操作码扩展技术

❖ 【例6.2】某计算机机器字长16位，指令系统有单字指令和双字指令两种格式



❖ （1）该机指令系统中最多有多少条指令？其通用寄存器堆最多有多少个寄存器?

❖ （2）双字指令的第二个源操作数，可以有多少种寻址方式?

❖ （3）假设单字指令的操作码最高两位为00，则单字指令最多有多少条？双字指令最多有多少条?

❖ （4）接上题，如果主存的代码段中有连续的三个字：561CH、90ABH、211DH，请分析它们是几条指令？是单字还是双字指令？试解释各指令所使用的寄存器及寻址方式编码。

❖ SR_1、SR_2：源操作数的寄存器号

❖ DR：目的操作数的寄存器号

❖ MOD：寻址方式码

❖ 单字指令：

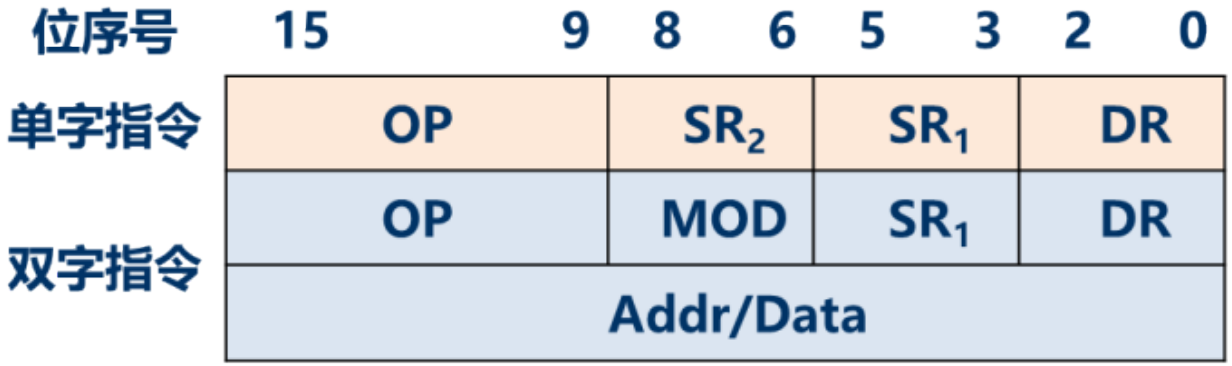
- （SR_1）op（SR_2）→DR
- Reg[SR_1] op Reg[SR_1] →Reg[DR]

❖ 双字指令

- 第一个源操作数：SR_1
- 第二个源操作数：由MOD和第二字共同定义
- 目的操作数：DR

2、操作码扩展技术

❖ 【例6.2】



❖ （1）该机指令系统中最多有多少条指令？其通用寄存器堆最多有多少个寄存器？

- 操作码字段OP：定长编码，固定7位
- 指令系统中最多有2^7=128条指令
- SR_1、SR_2和DR寄存器字段：3位
- 能对2^3=8个寄存器编码，故GR最多有8个

❖ （2）双字指令的第二个源操作数，可以有多少种寻址方式？

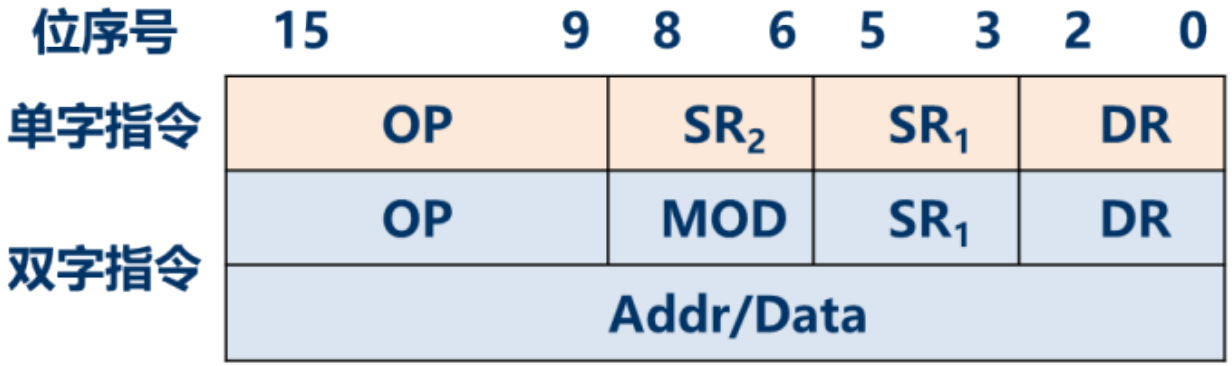
- MOD字段3位，故有2^3=8种寻址方式

❖ （3）假设单字指令的操作码最高两位为00，则单字指令最多有多少条？双字指令最多有多少条？

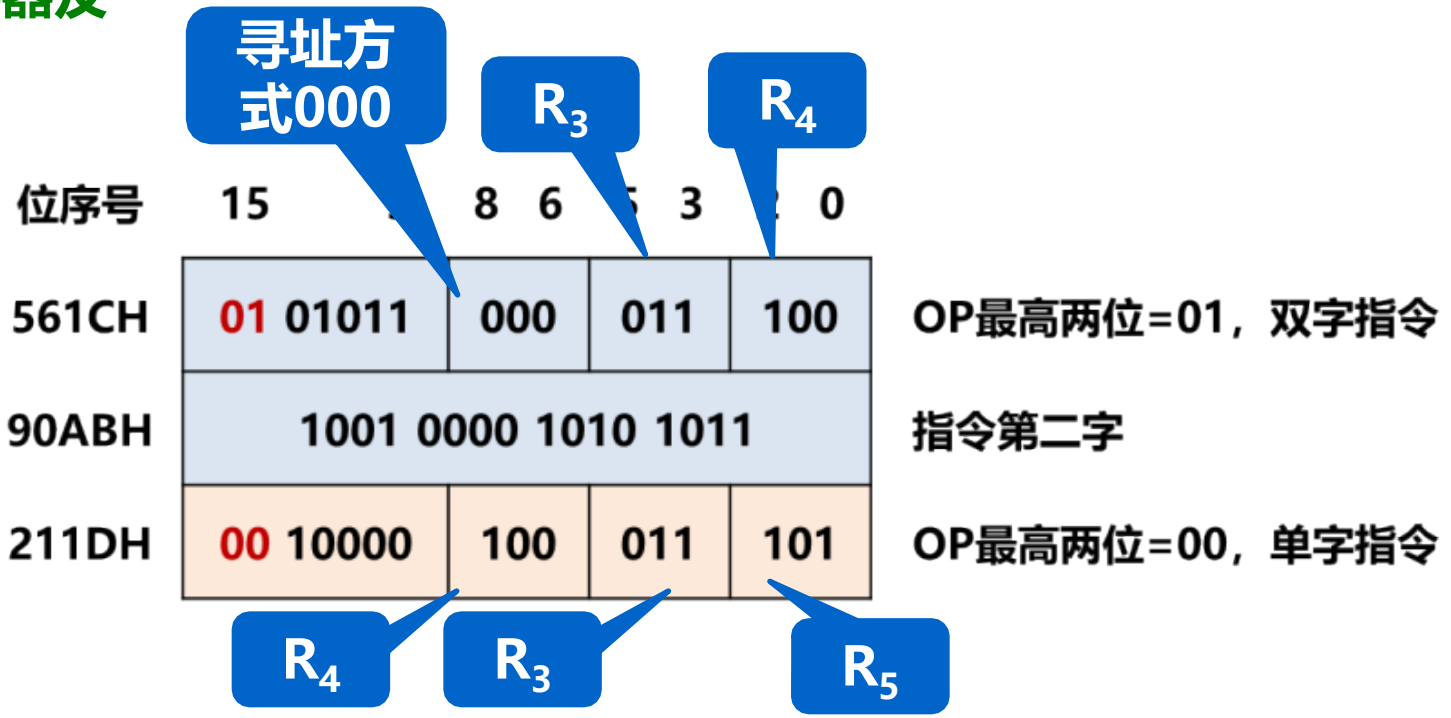
- 单字指令操作码字段：=00×××××
- 高2位为00，低5位可以有2^5=32个编码
- 单字指令：最多有32条
- 双字指令：最多有128-32=96条

2、操作码扩展技术

- 【例6.2】
- （4）接上题，如果主存的代码段中有连续的三个字：561CH、90ABH、211DH，请分析它们是几条指令？是单字还是双字指令？试解释各指令所使用的寄存器及寻址方式编码。



- 将指令字解码
  - OP的最高两位=00：单字指令
  - OP的最高两位≠00：双字指令



2、操作码扩展技术

❖RISC-V指令系统：

- 支持多种指令字长：16位、32位、48位、64位，分属于不同的指令子集
- 操作码opcode：指令字的低位，指定各种指令字长
- 32位架构：opcode位于低7位

位序号	……32	31	16	15	0	指令字长
				xxxx xxxx xxxx xxaa		aa≠11：16位
		xxxx xxxx xxxx xxxx		xxxx xxxx xxxbbb11		bbb≠111：32位
	……xxxx	xxxx xxxx xxxx xxxx		xxxx xxxx xx011111		48位
	……xxxx	xxxx xxxx xxxx xxxx		xxxx xxxx x0111111		64位
	……xxxx	xxxx xxxx xxxx xxxx		xnnn xxxx x1111111		nnn≠111：80+16*n位
	……xxxx	xxxx xxxx xxxx xxxx		x111 xxxx x1111111		为≥192位保留
字节地址	base+4	base+2		base		

6.3 寻址方式

- **定义：确定本条指令的操作数地址，以及下一条将要执行的指令地址的方法。**
  - 寻址方式与硬件结构紧密相关，由硬件完成寻址过程。
  - 直接影响指令格式和指令功能。
  - 对程序设计的效率、程序占用存储空间等方面都起着重要作用。
- **为何引入寻址方式?**
  - 提高编程的灵活性与程序效率
  - 压缩程序占用的存储空间
  - 扩大指令访问的存储空间



6.3.1 指令寻址

6.3.2 数据寻址

6.3.1 指令寻址

- 指令寻址：如何确定下一条指令在主存中的地址
- 指令地址：总是由程序计数器PC来指示，PC的内容就是指令在主存中的地址
- 1、顺序寻址方式
- 指令在主存中按顺序存放，计算机工作过程：
  - 取指令：根据PC从存储器取出当前指令（存入指令寄存器IR），PC自动+1/+n；
  - 分析、执行这条指令
  - 取下一条指令…
- 指令顺序寻址：通过PC自增，自动形成下一条指令的地址，置入PC



6.3.1 指令寻址

- 2、跳跃寻址方式：
- 通过转移类指令实现：程序不是顺序执行时，需要通过执行一条转移指令，跳跃到不连续的目标地址（Target Address，TA）继续执行
  - 地址码字段：转移相关信息→目标地址TA
  - 指令执行：TA→PC
  - 转移指令执行完后，控制器就按PC中的新地址去取下一条指令，从而实现转移
- 4种转移方式得到转移目标地址TA
  - （1）直接转移：TA=A；
  - （2）相对转移：TA=（PC）+A；A是补码表示的偏移量，可正可负
  - （3）寄存器寻址的间接转移：TA=Reg[Ri]（简记为TA=（Ri））
  - （4）寄存器相对转移：TA=Reg[Ri]+A（简记为TA=(Ri)+A）
    - 也称：变址寻址方式的转移（x86），基址寻址方式的转移（RISC-V）



6.3.2 数据寻址

- 形式地址：指令的地址码字段A，通常都不代表操作数的真实地址。
- 有效地址：指操作数在存储器中的真实地址，记作EA，它是由寻址方式和形式地址共同来确定的。

- 常见的有9种基本的寻址方式
  - 复合寻址：基本寻址方式的组合
- 所有的计算机CPU均采用多种寻址方式
  - 问题：如何识别是哪种寻址方式?



6.3.2 数据寻址（以单操作数/单地址为例）

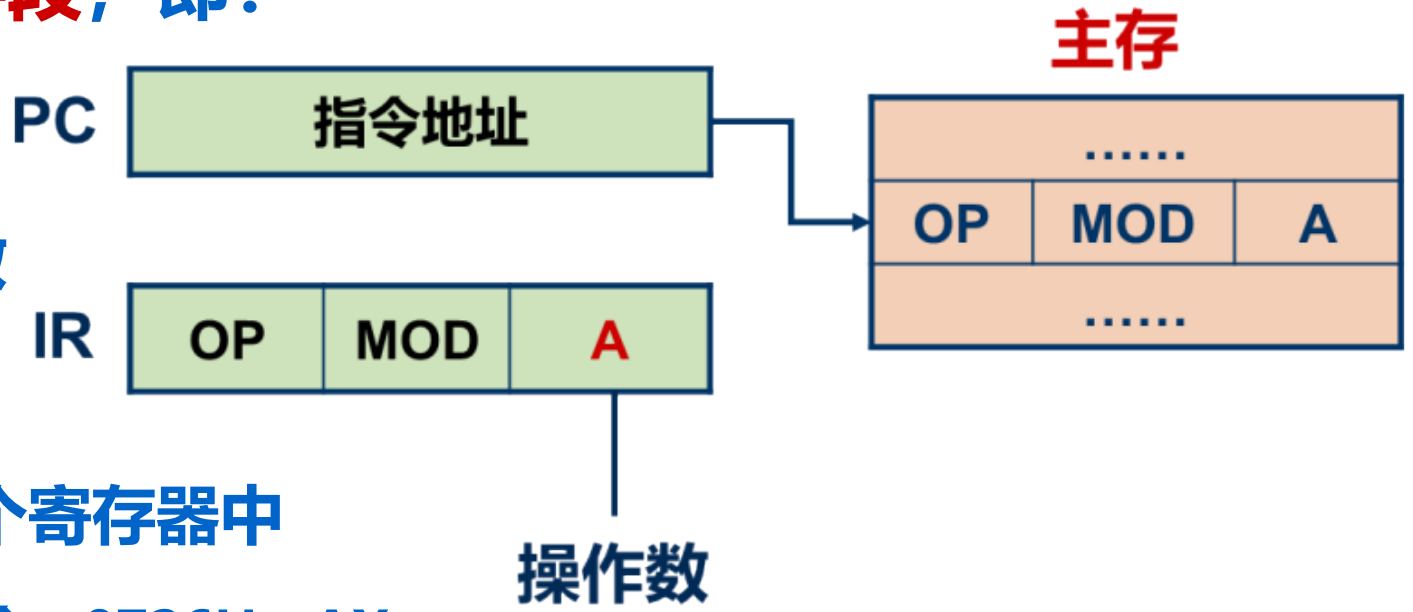
1. 立即寻址
2. 直接寻址
3. 间接寻址
4. 寄存器寻址方式
5. 寄存器间接寻址方式
6. 变址寻址
7. 基址寻址
8. 相对寻址
9. 堆栈寻址
10. 复合寻址

总结



1、立即寻址(Immediate Addressing)

- **操作数：在指令的地址码字段，即：**
  - Operand=A
- **取出指令，就立即得到了操作数**



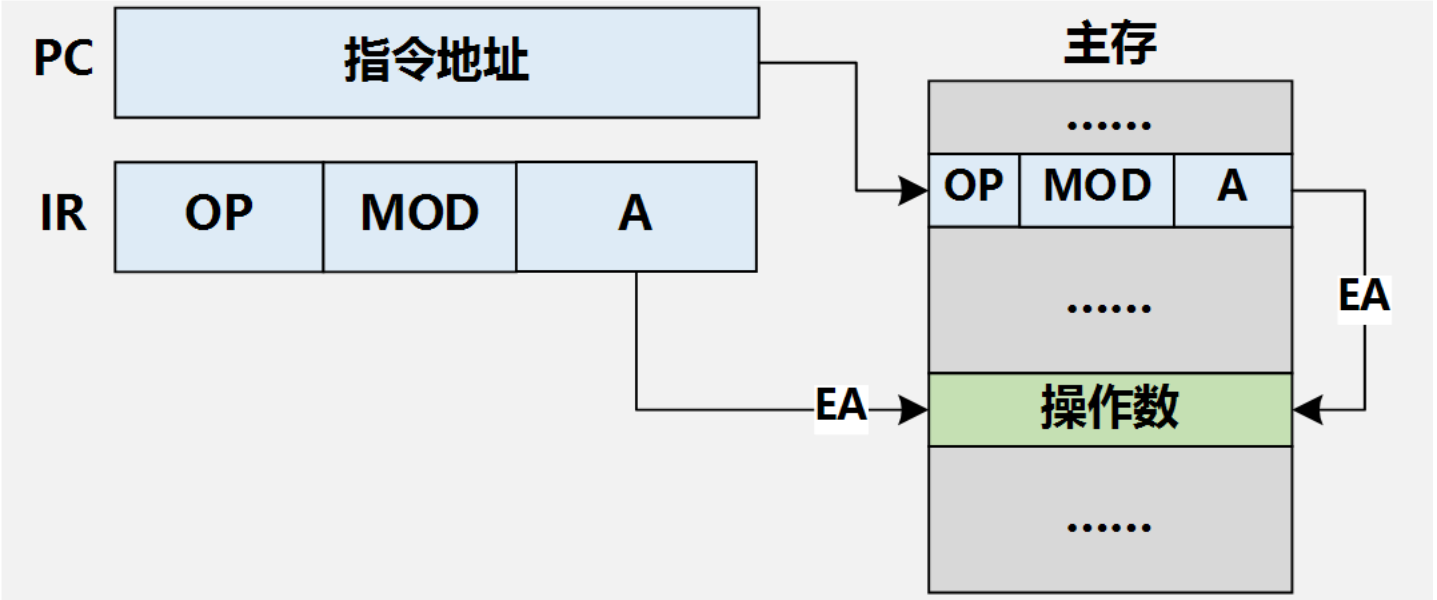
- **例如：将立即数0726H送到某个寄存器中**
  - MOV AX, 0726H ;x86指令：0726H→AX
  - li t0, 0x726 #MIPS32伪指令，等价于addi t0,$0,0x726：0726H→t0
  - li t0, 0x726 #RISC-V伪指令，等价于addi t0,x0,0x726：0726H→t0
  - ADDI x2,x31,#0x726;ARM指令：0726H→x2

2、直接寻址(Direct Addressing)

- 操作数：位于存储器中，操作数所在的存储器单元的地址存放在指令的地址字段A中
  - EA = A
  - Operand = Mem[EA] =Mem[A]
  - 简记为：Operand = (A)

- 例如：
  - MOV AX,[1000H]
  - ADD [2000H],BX

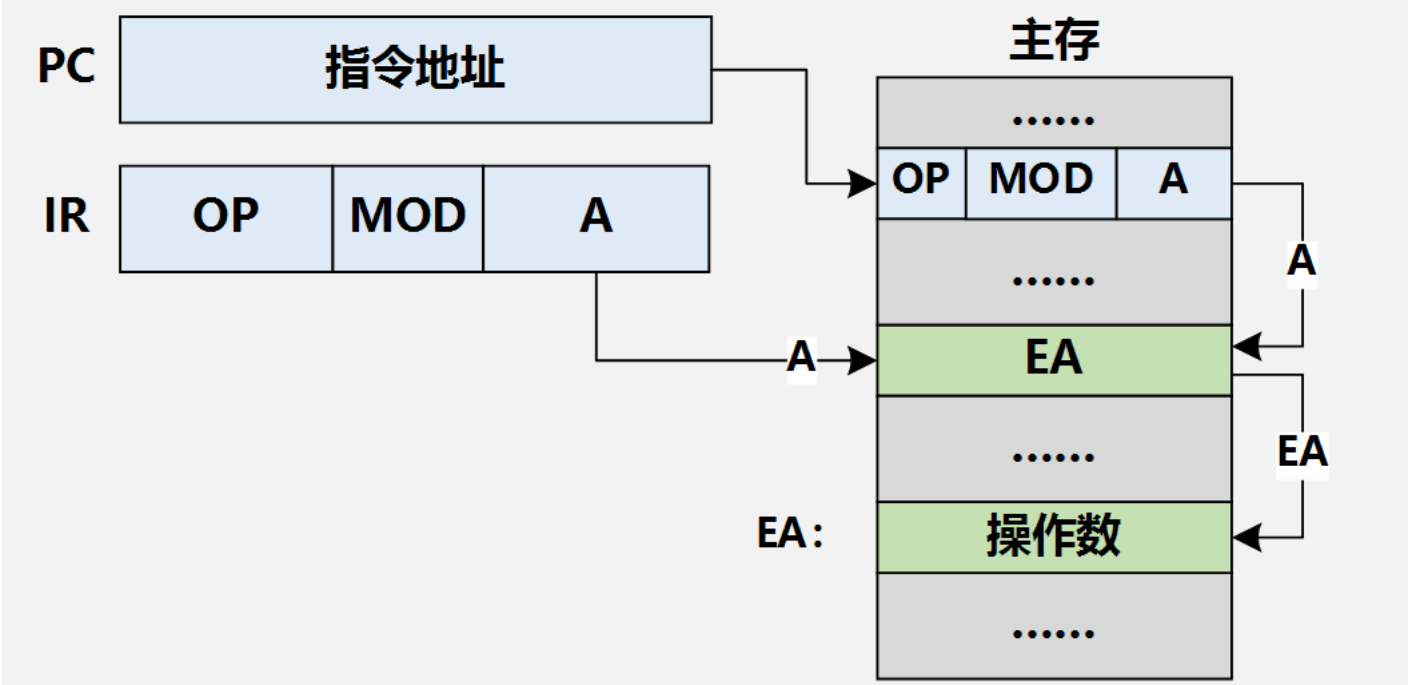
- 取出指令，就得到了EA
- 还需访问主存1次，才能得到操作数



3、间接寻址(Indirect Addressing)

❖操作数：位于存储器中，操作数所在的存储器单元地址也存放在存储器中，该存储器地址则存放在指令的地址字段A中，即：

- EA = Mem[A]
- Operand = Mem[EA]
  =Mem[Mem[A]]
- 简记为：EA=(A)
  Operand = ((A))
- 即：A为操作数地址的地址

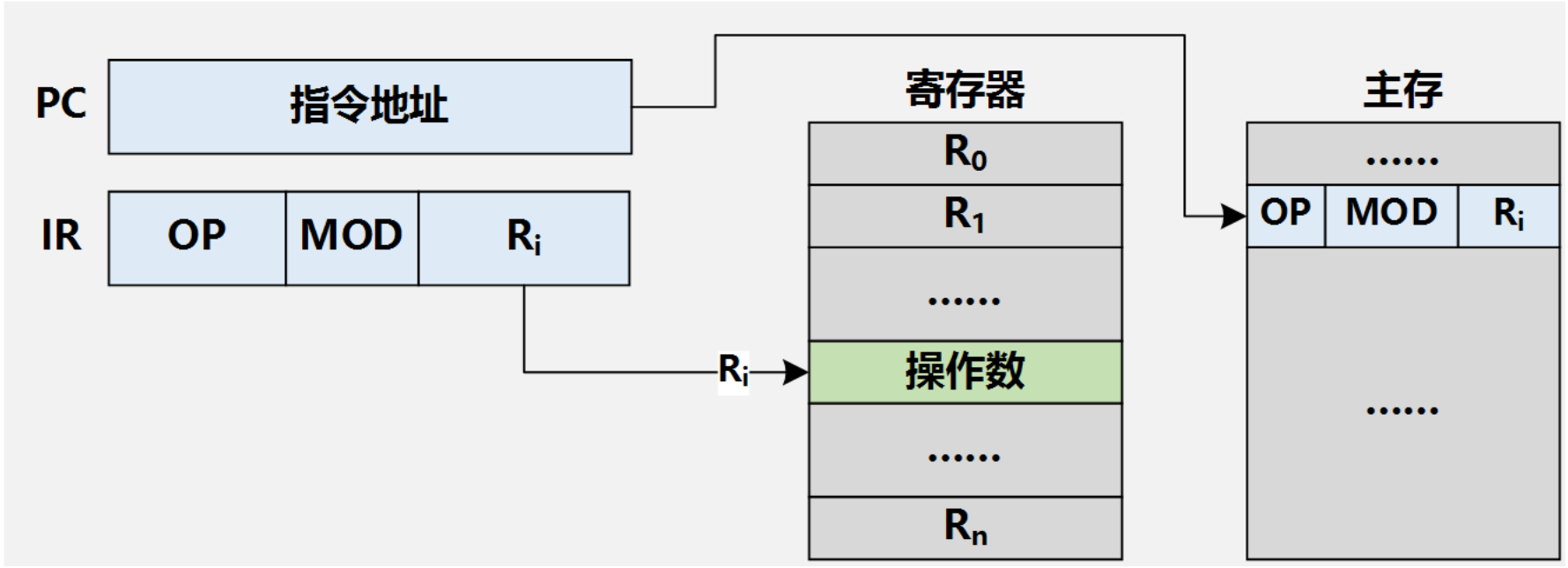


❖取出指令，就得到了EA在主存的地址，还需访问主存2次，才能得到操作数。

- 第一次：得到EA
- 第二次：得到操作数

4、寄存器寻址方式

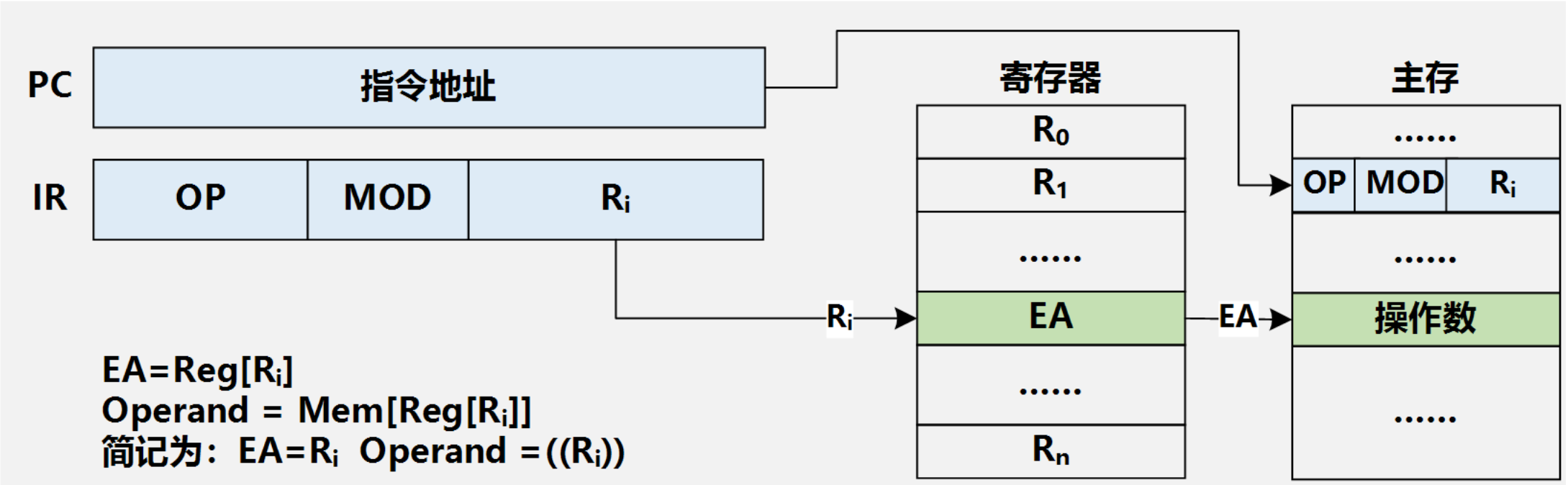
- **操作数：位于寄存器中，操作数所在的寄存器编号存放在指令的地址字段A中，即：**
  - Operand = Reg[R_i]
  - 简记为：Operand = (R_i)



- **例如：**
  - MOV AX, BX
  - MOV AX,0726H
  - addi t0,t1,100
  - xor s0,t1,t2
  - ADDI w2,w1,#100

5、寄存器间接寻址方式

❖操作数：位于存储器中，操作数所在的存储器地址存放在寄存器中，而该寄存器编号存放在指令的地址字段A中，即：



- DATA = （EA）
- EA = （R_i）

❖例如：

- ADD AX, [BX]

变址寻址、基址寻址和相对寻址

❖ 相同点：

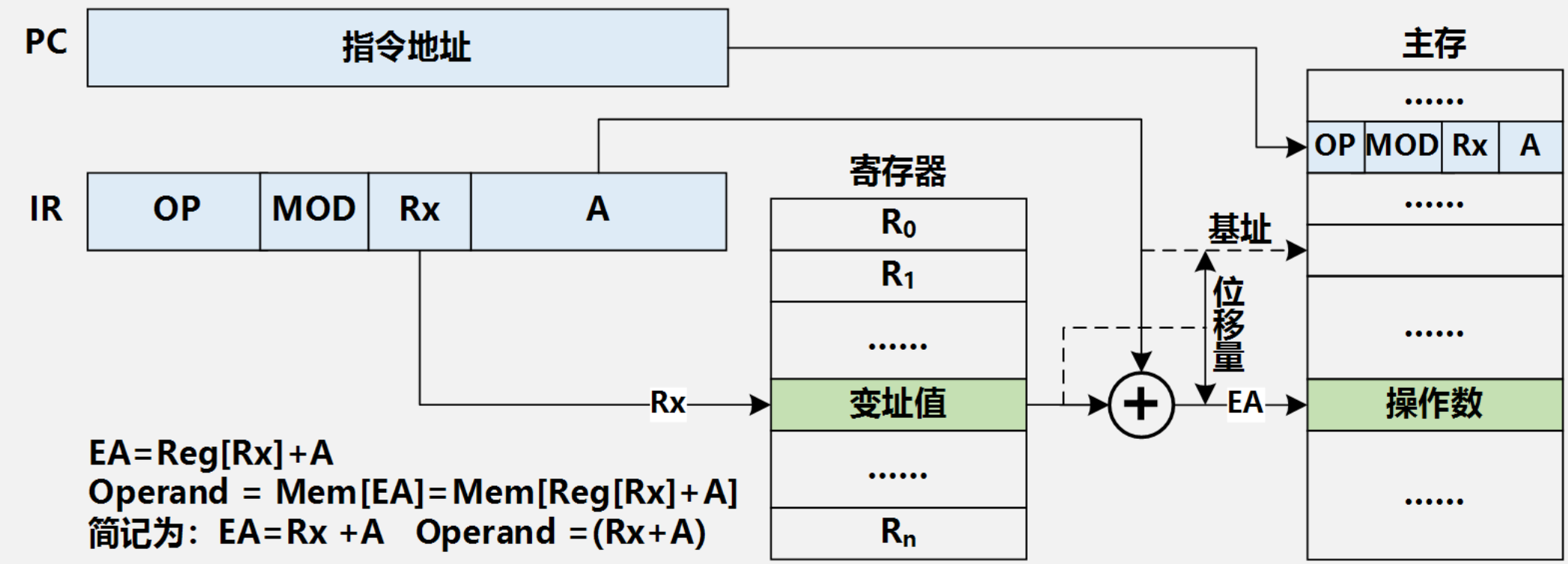
- 操作数：位于存储器中；
- EA=（寄存器）+A

❖ 不同点：

- 使用的寄存器不同：
  - 变址寻址：EA=（RI）+A，变址寄存器
  - 基址寻址：EA=（Rb）+A，基址寄存器
  - 相对寻址：EA=（PC）+A，程序计数器
- 应用场合不同：
  - 变址寻址：数组、字符串、向量等成组数据的访问；
  - 基址寻址：系统级用户；或者二维数组
  - 相对寻址：转移指令的相对跳转

6、变址寻址（Indexed Addressing）

❖操作数：位于存储器中，操作数所在的存储器地址EA由变址寄存器RI和指令的地址字段A指出：



- DATA=（EA）
- EA=（RI）+A

6、变址寻址（Indexed Addressing）

- 变址寻址通常用来对字符串处理、数组运算等成组数据处理。
- 字符串处理：
  - 字符串在主存的首地址A：由指令的地址码字段指出，作为基准地址；
  - 字符串的字符序号：变址寄存器的内容。
- 数组运算：
  - 数组首地址（第0个元素地址）：作为指令的地址码
  - 数组元素的序号（下标）：变址寄存器内容

6、变址寻址（Indexed Addressing）

- 【例6.3】下面是一段x86的汇编语言源程序，使用变址寻址构造了一个循环：
- （1）阅读分析后，简述其功能。
- （2）判断x86 CPU是按字编址还是按字节编址？说出理由。
- （3）写出对应的C语言程序片段。

```
.data
        num    dw  1,5,7,28,97,100,-23,89,65,-45;定义10个元素的数组，每个元素16位
.code
        MOV    ESI,0
        MOV    AX,0            ;0→AX
        MOV    CX,10           ;10→CX
L:      ADD    AX, num[ESI]    ;AX+Mem[ESI+&num]→AX
        INC    ESI             ;ESI+1→ESI
        INC    ESI             ;ESI+1→ESI
        LOOP   L               ;CX-1→CX，if (CX≠0) goto L
```

数组首址

变址寄存器

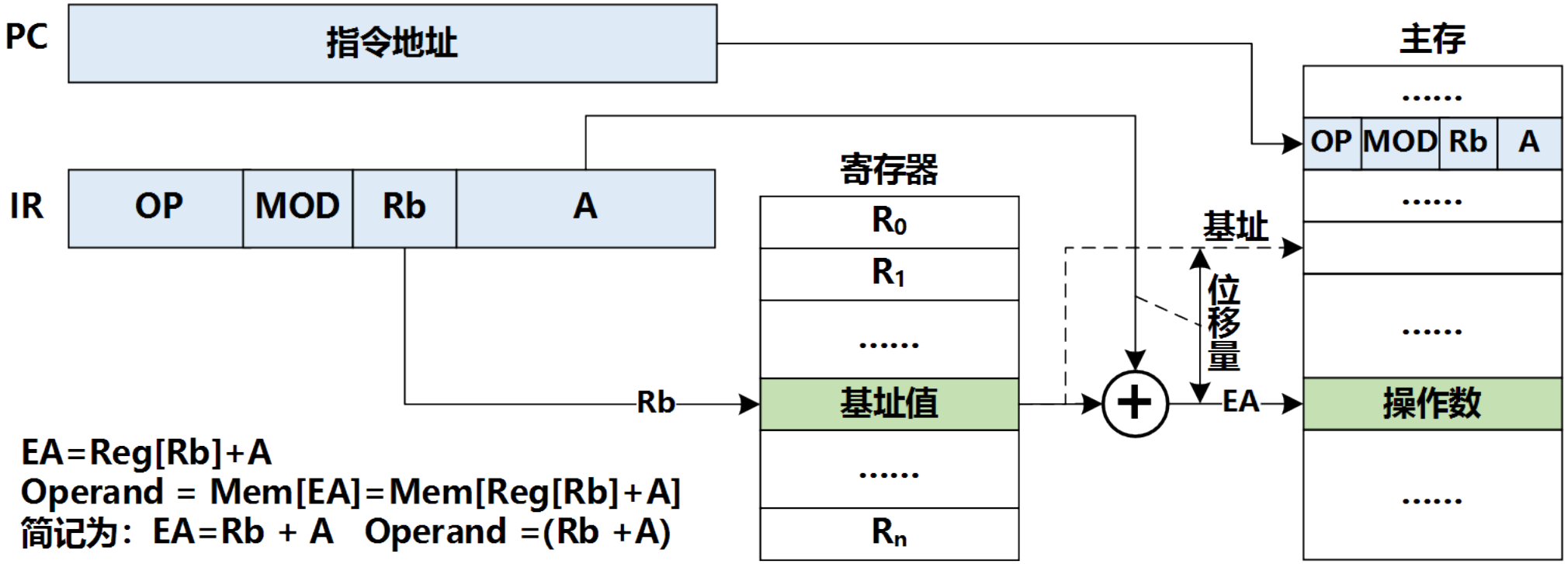
6、变址寻址（Indexed Addressing）

- 【例6.3】
- （1）程序功能：将数组num的10个数据累加，累加和在AX中。
- （2）num数组中每个数据都为16位，每累加一个数据，变址指针ESI都执行+2操作以指向下一个数据，可推断：16位数据占据2个地址编码，故x86是按照字节编址的
- （3）循环有固定的循环次数（10次），可以用C语言的for语句描述

```
int num[]= {1,5,7,28,97,100,-23,89,65,-45};
int i,sum=0;
for (i=0;i<10;i++)
        sum+=num[i]
```

7、基址寻址（Based Addressing）

❖操作数：位于存储器中，操作数所在的存储器地址EA由基址寄存器Rb和指令的地址字段A指出：



- DATA = （EA）
- EA = （Rb） + A

7、基址寻址（Based Addressing）

- **起源：多用户计算机系统**
  - 当操作系统为多道程序分配主存空间，给每个用户一个基地址并放入其相应的基址寄存器
  - 在程序执行时，以基址为基准自动进行逻辑地址到物理地址的变换
  - 例如：页式和段式虚拟存储器的地址变换
- **访问指令：程序执行具有局部性，即在某一段时间内只访问部分存储区域**
  - 基址寄存器：字长应足以指向整个主存空间；位移量：只要能寻址本区域即可
  - 既能缩短指令长度，又可以扩大寻址范围
- **访问数据：访问主存中的成组数据**
- **与变址寻址很难区分，对比：**
  - 基址寄存器的内容是基准量，指令中的地址码字段A是偏移量，因此其位数较短
  - 应用场合：基址寻址面向系统，可以解决程序在主存中的重定位和扩大寻址空间等问题；而变址寻址是面向用户编程，用来访问字符串、向量和成批数据

7、基址寻址（Based Addressing）

- X86：将基址或者变址寄存器内容+偏移量产生有效地址的寻址方式，统一称为**寄存器相对寻址**（Register Relative Addressing）
- MIPS32、RISC-V和ARM中：称这种寻址为基址寻址

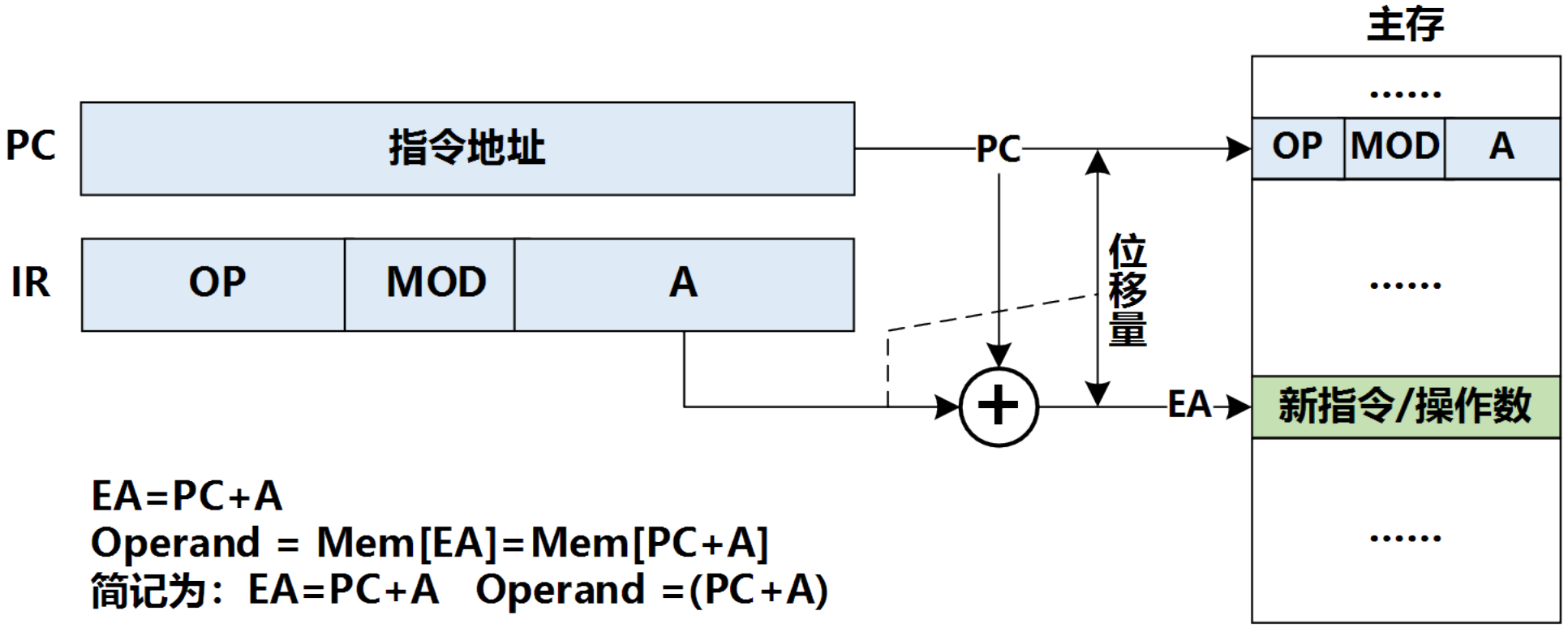
```
lw      s0, 100(t0)          # MIPS32/RISC-V指令：Mem32[t0+100]→s0
LDR   R1, [R2,#16]           ;ARM指令：Mem[R2+16]→R1
```



8、相对寻址（Relative Addressing）

- **操作数：位于存储器中，操作数所在的存储器地址EA由程序计数器PC和指令的地址字段A指出：**
  - DATA = （EA）
  - EA = （PC） + A

```
j L       #RISC-V指令：PC+offset20*2→PC;
beq  $2,$3,L   #MIPS指令：PC+4+offset16*4→PC
LDR R1, =DataL  ;ARM指令：Mem[PC+8+offset12]→R1
```



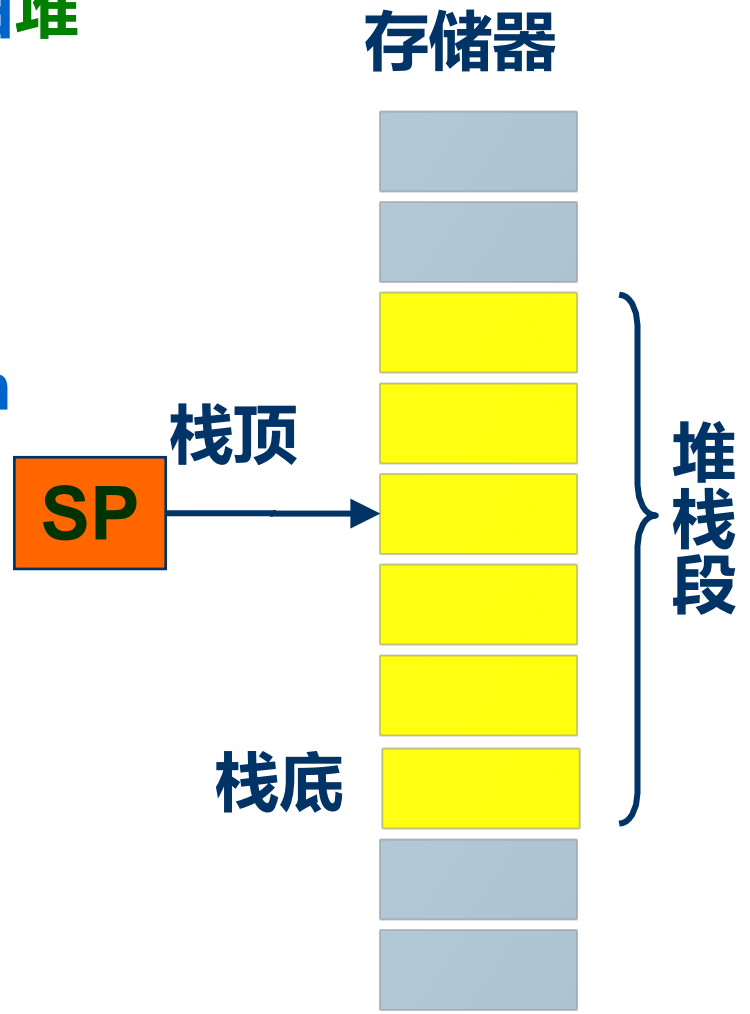
EA=PC+A
Operand = Mem[EA]=Mem[PC+A]
简记为：EA=PC+A　Operand =(PC+A)

8、相对寻址（Relative Addressing）

- A：相对偏移量DISP，可正可负，用补码表示，相对PC向后或向前转移/访问
- 主要用于转移指令，转移指令的功能是修改PC的值为EA，即：PC+A→PC
- 汇编程序中的转移目的地址：以标号标识，翻译成机器指令时，计算偏移量的工作是由汇编器自动完成的——偏移量 = 转移目的地址-PC
  - 计算偏移量时，PC值：使用当前指令地址，还是使用下一条指令的地址?
  - 多数计算机：在执行当前指令时，PC已经自增指向下一条指令，以下一条指令地址为基准
  - RISC-V：使用当前指令地址，方便计算机流水线实现时，只记录PC的值
- 相对寻址：便于实现程序的浮动存储，程序无论装入主存何处，均能正确运行

9、堆栈寻址（Stack Addressing）

- **操作数：位于存储器中，操作数所在的存储器地址EA由堆栈指针寄存器SP隐含指出，通常用于堆栈指令**
  - DATA =（EA）
  - EA =（SP）
- **堆栈：由若干个连续主存单元组成的先进后出（first in last out，即FILO）存储区**
- **栈底：第1个放入堆栈的数据所存放的单元**
  - 固定不变的，地址一般由堆栈段的段寄存器指出
- **栈顶：最近放入的数据所存放的单元**
  - 随着数据的入栈和出栈而发生变化，地址由堆栈指针SP指明



- **堆栈寻址：主要用来暂存中断和子程序调用时现场数据及返回地址**

堆栈的结构

递减堆栈

- **上推堆栈：栈底地址≥栈顶地址**
  - 堆栈从高地址(栈底)向低地址(栈顶)扩展
  - 多数计算机中

递增堆栈

- **下推堆栈：栈底地址≤栈顶地址**
  - 堆栈从低地址(栈底)向高地址(栈顶)扩展
  - 少数计算机

结构?

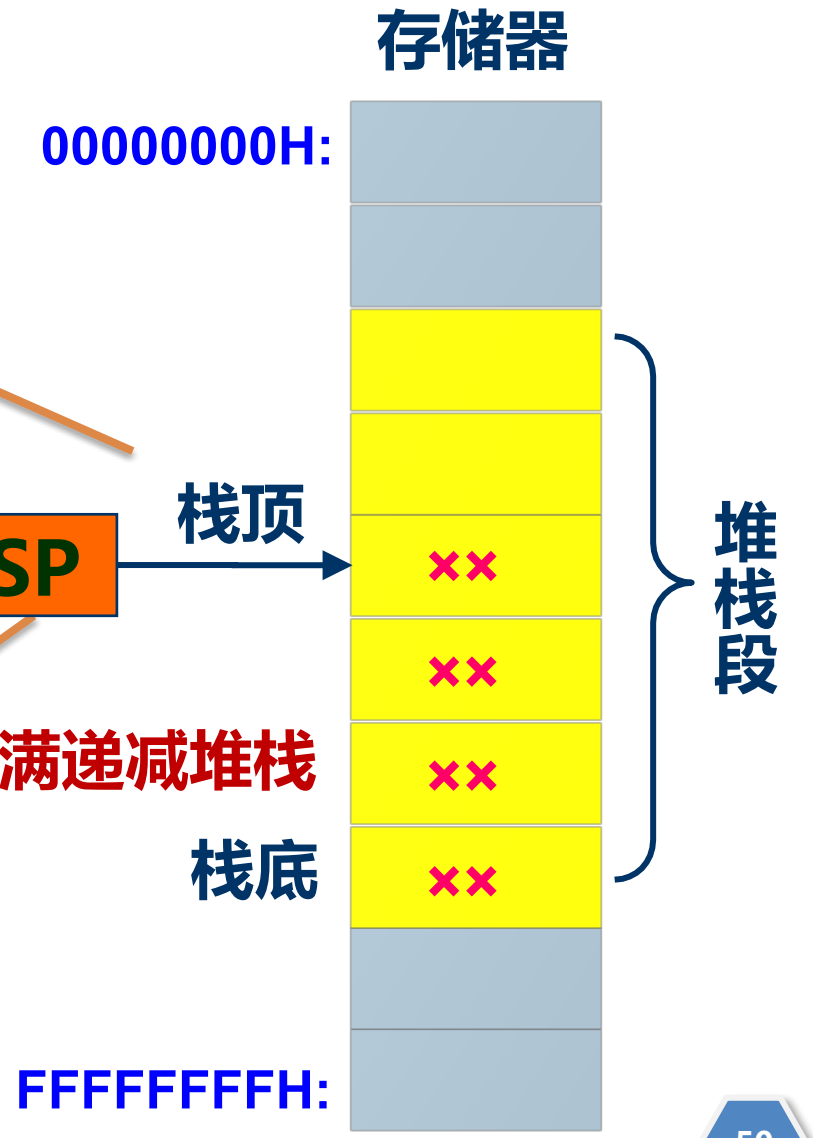
SP管理办法?

- **堆栈指针的管理：两种方法**
  - SP总是指向栈顶最后压入的有效数据

满堆栈

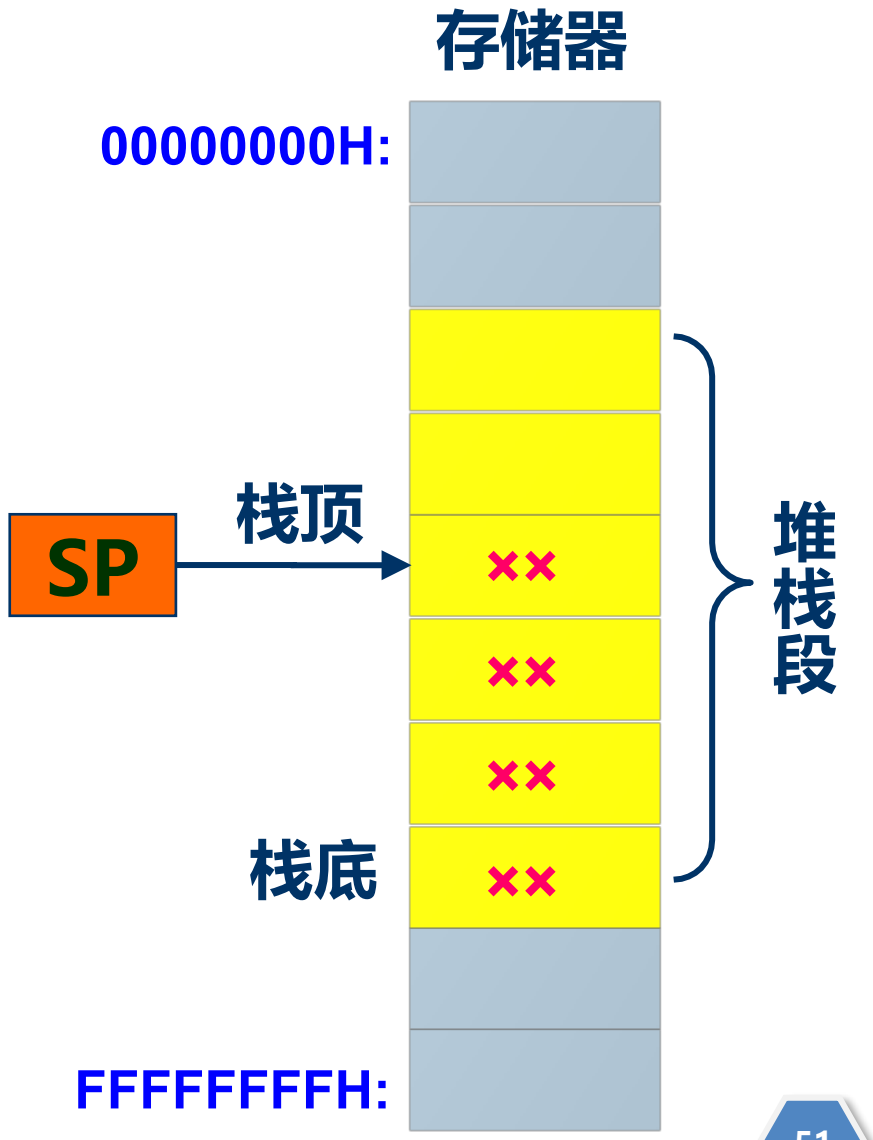
  - SP总是指向栈顶的空单元

空堆栈



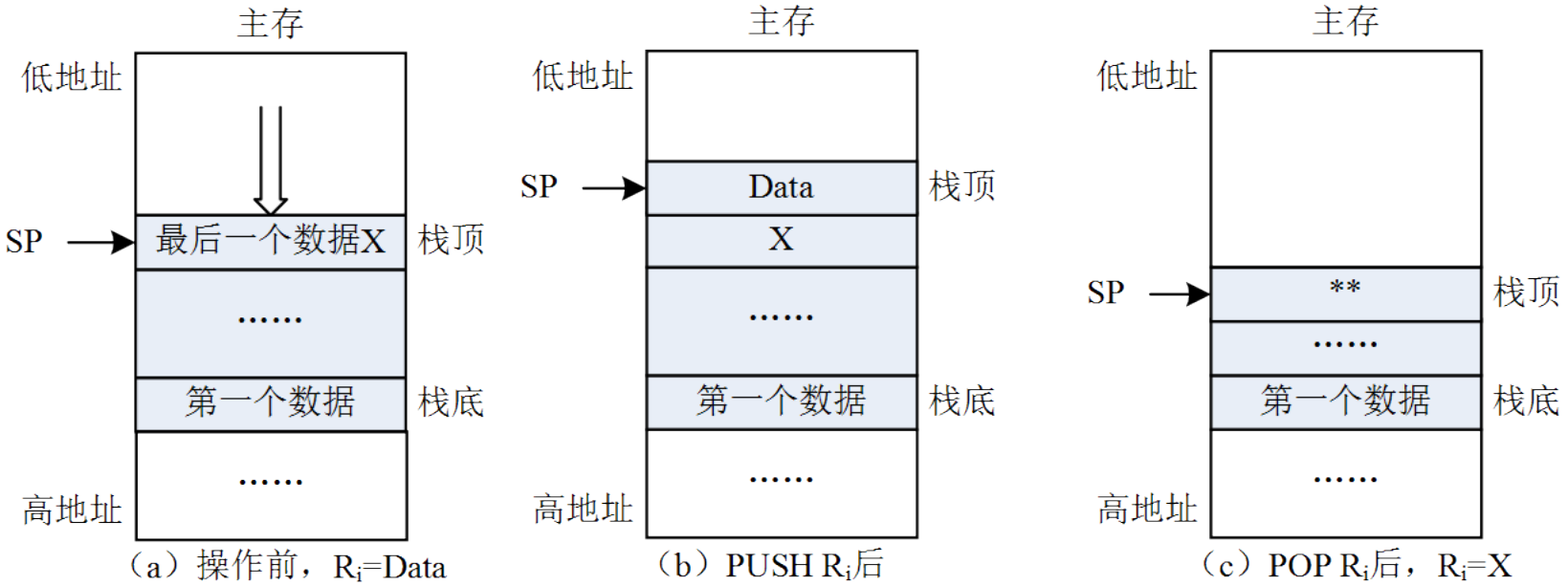
对堆栈的访问与操作

- 只对栈顶操作
- 两种操作：
  - 压入：PUSH指令；数据存入堆栈
  - 弹出：POP指令；从堆栈取出数据
- 假设图示堆栈：满递减堆栈
  - 上推堆栈；
  - SP指向栈顶有效单元；
  - 数据字长为1B，按字节编址；
  - PUSH和POP操作，做了什么?



堆栈指令的实现

- **压入指令 PUSH Ri：将Ri寄存器内容压入堆栈**
  - (SP) - 1→SP，(Ri) →(SP)
- **弹出指令POP Ri：从堆栈中弹出1个数据送Ri**
  - ((SP)) → Ri ,(SP) + 1→SP



- **思考：**
  - 指令的两个操作，能否顺序互换?
  - 变成下推堆栈（递增堆栈）？
  - SP指向空单元（空堆栈）？
  - 如果Ri是16位?

10、复合寻址

- 对于一台具体的计算机：可能
  - 只采用其中几种寻址方式
  - 将这些基本寻址方式作某些变化
  - 将几种基本寻址方式组合起来使用，形成一种新的寻址方式→复合寻址
- 用于访问二维数组：
  - 位移量：指向二维数组的首址
  - 基址寄存器：指向行地址
  - 变址寄存器：指向行中的元素

- 基址变址寻址：
  - 操作数：在存储器中；指令中指定一个基址寄存器和一个变址寄存器，指令中的地址码给出位移量
  - 有效地址EA：三者相加而成
    - DATA=（EA）
    - EA=（Rb）+（RI）+A

```
MOV  AX,num[BP][SI];        Mem[BP+SI+&num]→AX
MOV   [BX][DI], AX;         AX→Mem[BX+DI]
LDR R1, [R2, #16]!;         ARM指令：Mem[R2+16]→R1，R2+16→R2
```

总结

寻址方式	操作数类型	地址码字段A	EA
立即寻址	立即数	立即数Data	——
直接寻址	存储器	直接地址A	EA=A
间接寻址	存储器	间接地址A	EA=(A)
寄存器寻址	寄存器	寄存器号n	——
寄存器间接寻址	存储器	寄存器号n	EA=（Rn）
变址寻址	存储器	形式地址X	EA=（RI）+X
基址寻址	存储器	相对偏移量Disp	EA=（Rb）+Disp
相对寻址	存储器	相对偏移量Disp	EA=（PC）+Disp

举例

- 【例6.4】某8位计算机指令格式：
  - DR为目的寄存器号
  - MOD为寻址方式码字段
  - 指令第二字为地址、数据或偏移量
  - 源操作数：由MOD字段和指令第二字共同确定
  - 单字指令：HALT、INC DR、DEC DR指令
  - 双字指令：其他指令
  - 跳转指令：由源操作数指出跳转地址

OP（4位）	DR（2位）	MOD（2位）
ADDR/ DATA / DISP		

MOD	寻址方式	DR	寄存器
00	立即寻址	00	R0
01	直接寻址	01	R1
10	变址寻址（SI）	10	R2
11	相对寻址	11	R3（SI）

指令助记符	OP
ADD	0000
SUB	0001
ADC	0010
SBB	0011
MOV	0100
INC	0101
JMP	1000
……	……
HALT	1111

内存地址的部分单元内容如下：

单元地址	内容	单元地址	内容	单元地址	内容
10H	83H	20H	40H	24H	11H
11H	90H	21H	23H	25H	12H
12H	10H	22H	02H	26H	F0H
13H	11H	23H	03H	27H	20H

（1） 若（PC）＝20H，变址寄存器（SI）＝10H，则此时启动程序执行，问执行了几条指令程序停止？写出每条指令的助记符、寻址方式、EA、操作数和执行结果。

（2）写出以下程序段的机器码。

```
    MOV   SI,#0    ;0→SI
    MOV   R0,#0    ;0→R0
L:  ADD   R0,[SI+80H];R0+Mem[SI+80H]→R0
    INC   SI       ;SI+1→SI
    JMP   L        ;goto  L，即&L→PC
```

（1）指令码分析：

- ❖ 当前指令在哪里？
  - 主存
- ❖ 主存哪个单元？
  - PC指定单元地址
- ❖ PC=20H，指令码？
  - 40H
- ❖ 单字双字？
  - 解析后确定

单元地址	指令码	助记符	寻址方式	EA	操作数	执行结果
20H	40H	MOV R0,23H	立即数	—	23H	(R0) = 23H
21H	23H					
22H	02H	ADD R0, [SI + 3H]	变址寻址	(SI) + 3H = 13H	(13H) = 11H	(R0) = 23H + 11H = 34H
23H	03H					
24H	11H	SUB R0, [12H]	直接寻址	12H	(12H) = 10H	(R0) = 34H - 10H = 24H
25H	12H					
26H	F0H	HALT	—	—	—	—

（2）写出程序的机器码

序号	程序助记符	功能
1	MOV SI,#0	0→SI
2	MOV R0,#0	0→R0
3	L：ADD R0，[SI+80H]	(R0)+(SI+80H) →R0
4	INC SI	SI+1 →SI
5	JMP L	L→PC

地址	指令码
00H	4CH
01H	00H
02H	40H
03H	00H
04H	02H
05H	80H
06H	5CH
07H	81H
08H	04H

直接寻址

如果相对寻址

6.4 指令系统的设计

- 6.4.1 指令系统的设计要求
- 6.4.2 指令类型
- 6.4.3 指令系统的发展历程
- 6.4.4 CISC与RISC特点

6.4.1 指令系统的设计要求

- 1、完备性：指令系统直接提供的指令足够使用，而不必用软件来实现。
- 2、有效性：是指利用该指令系统所编写的程序能够高效地运行。
  - 程序占据存储空间小、执行速度快。
- 3、规整性：
  - 对称性：所有的指令都可使用各种寻址方式
  - 匀齐性：指令可以支持各种数据类型
  - 一致性：指令和数据的格式一致、长度有一定的关系，以方便处理和存取
- 4、兼容性：“向上兼容”，即低档机上运行的软件可以在高档机上运行

需要通过“折中”实现最优的设计



6.4.2 指令类型

- 1、数据传送类
- 2、算术逻辑运算指令
- 3、移位操作指令
- 4、程序控制类指令
- 5、堆栈操作指令
- 6、输入输出指令
- 7、处理器控制类指令
- 8、特权指令

- 1. 数据传送指令
- 源和目的操作数之间的传送：一个/一批
- Intel X86使用助记符：MOV
- RISC CPU：
  - 装立即数：立即数→寄存器
    - move/li伪指令
  - 装数/取数：存储单元→寄存器
    - lw/ld（load data from memory to register)
  - 存数：寄存器→存储单元
    - sw/st（store a register data to memory)
- 数据交换指令SWAP：互换，实现双向数据传送

6.4.2 指令类型

- 2．算术逻辑运算指令
  - 算术运算：加、减、乘、除等：ADD、SUB、MUL、DIV
  - 逻辑运算：与、或、非、异或等：AND、OR、NOT、XOR
  - 位操作指令：不一定有
    - 位测试：测试指定位的值（譬如X86的TEST）
    - 位清零：某些位清零（譬如ARM的BIC）
- 3． 移位操作指令
  - 算术移位、逻辑移位：可以实现简单的乘法和除法运算
  - 循环移位：不一定有

6.4.2 指令类型

❖ 4．程序控制类指令

- 用于控制程序的流向，使程序具有测试、分析判断、决策的能力，有四种：
  - 无条件转移
  - 条件转移
  - 过程调用与返回
  - 陷阱（Trap）
- 共同点：修改PC的值，发生转移或者分支

❖ （1）无条件转移指令：无条件转至目的地址处执行

- 助记符：JMP（X86 CPU）/J（RISC-V）
- 操作：有效转移地址（目的地址）→PC
  - JMP X：将指令地址无条件转至X处，X→PC

6.4.2 指令类型

- **4．程序控制类指令**
- **（2）条件转移指令：条件满足，转至目的地址处执行，否则顺序执行**
  - 助记符：Jcc/Bcc
  - cc：条件码，零标志ZF、溢出标志OF、进位标志CF等
  - 分支操作（Branch）：
    - cc=1：有效转移地址（目的地址）→PC
    - cc=0：空操作（指令结束，实际上是顺序执行）
  - 条件码的生成、转移的判断：可以在一条指令执行中完成，也可以在两条指令执行中完成。

条件转移指令举例

- 用一条转移指令完成
  - 完成比较运算→根据结果判断转移条件是否成立→确定是否转移
- RISC-V CPU举例：
  - beq t0,t1,L
  - 功能：if (t0==t1) goto L
  - 实际上CPU的操作：
    - PC=(t0==t1) ? (PC+disp) : (PC+4)

- 若用两条指令完成：
  - 前一条指令：算术运算指令，生成条件码
  - 第二条指令：转移指令，根据条件码判断是否转移。
- X86举例：
  - CMP AX，BX ;AX-BX，置标志位
  - JE L ;ZF=1，则转移到L处执行，否则不转移
  - 对JE指令，实际上CPU的操作：
    - IP = ZF ? (&L) : (IP+4)

6.4.2 指令类型

RISC-V、MIPS：jal

RISC-V、MIPS：jr

❖ 4．程序控制类指令

- （3）子程序调用、返回指令：
  - 调用指令CALL：从当前的程序位置转至子程序执行
    - ①PC压栈：PUSH PC
    - ②子程序入口地址→PC
  - 返回指令RETURN：子程序执行完后重新返回到原程序的断点。
    - 出栈→PC：POP PC
  - 有些CPU使用专用寄存器来存放返回地址
- （4） 陷阱指令、中断指令：INT n
  - 陷阱其实是一种意外事故的中断

子程序第一条指令的地址

RISC-V使用x1
MIPS使用$31

6.4.2 指令类型

❖5．堆栈操作指令

- 堆栈：用于保存中断和子程序调用时的现场数据和返回地址
- 堆栈操作指令：是一种特殊的数据传送指令，对SP指针指向的主存单元进行读写，并移动SP
- PUSH：入栈指令，把指定的操作数送入堆栈栈顶
- POP：出栈指令，把栈顶的数据取出，送到指令所指定的目的地。

❖6．输入输出指令：

- IN指令：外设端口→寄存器/存储单元
- OUT指令：寄存器/存储单元→外设端口

外设接口中的寄存器

- 不是所有CPU都有IN、OUT指令
  - 外设单独编址的计算机：设有输入输出指令
  - 外设和主存统一编址的计算机：无需设置输入输出指令，用访存指令访问外设

6.4.2 指令类型

❖7．处理器控制指令：

- HLT：暂停程序的执行，表现是不执行下一条指令；往往是等待中断（WAIT）
- NOP：处理器不执行任何操作，只对程序计数器计数
- 条件码操作指令：标志位进行置位（BTS）、清除（BTR）或者取反（BTC）操作。
- 开中断STI和关中断CLI：对中断允许位IF的置位或清除操作

❖8.特权指令：

- 在多用户、多任务的计算机系统中，必需要设置特权指令，用于系统资源的分配和管理
- 只能由操作系统或其他系统软件使用，普通用户没有使用权限

❖向量指令、多处理机指令

- x86的Pentium III处理器，SSE（Stream SIMD Extentions）指令集



6.4.3 指令系统的发展历程

- 简单指令系统（Simple Instruction Set）
  - 典型特征：硬件结构比较简单，指令系统条数少
  - 典型CPU：Intel 4004微处理器，含2250个晶体管，8个寄存器，指令字长8位，数据字长4位，指令系统有46条指令。
- 复杂指令系统（Complex Instruction Set）
  - 典型特征：指令格式不固定，寻址方式丰富，功能复杂，条数多
  - 典型CPU：Intel X86、VAX11/780（300+指令、16种寻址方式）
  - CISC：Complex Instruction Set Computer
  - 一些比较简单的指令，仅占指令总数的20%，但在程序中出现的频率却达到80%
  - 占指令总数20%的最复杂的指令，却占用了控制存储器容量的80%，且使用频率却不高。

6.4.3 指令系统的发展历程

❖精简指令系统： （Reduced Instruction Set）

- 典型特征：指令格式固定，寻址方式少，功能简单
- 典型CPU：MIPS、ARM、SPARC、RISC-V
- RISC：Reduced Instruction Set Computer

❖RISC体系结构的处理器经历了三代：

- 第一代：32位，支持Cache，软件支持较少，性能与CISC体系结构的产品相当，如RISC Ⅰ、MIPS、IBM801等。
- 第二代：提高了集成度，具有单指令流水线，可同时执行多条指令；增加了对多处理机系统的支持，提高了时钟频率，完善的存储管理体系、软件支持系统。如MIPS的R3000处理器。
- 第三代：64位，采用了超级流水线技术和超标量技术，提高了指令级的并行处理能力，使RISC处理器的整体性能更好。如MIPS的R4000处理器。



6.4.4 CISC与RISC特点

- CISC：Complex Instruction Set Computer，复杂指令集计算机
- RISC：Reduced Instruction Set Computer，精简指令集计算机

- 1、CISC的特点
- ①指令系统复杂：条数多(≥100条)、寻址方式多(≥5种)、指令格式多(≥4种)
- ②处理器
  - 采用微程序控制实现指令的执行；有较多的专用寄存器，通用寄存器少；
  - 指令执行需要经常访问存储器，故平均指令周期长，大多数指令需要多个时钟周期完成
- ③性能
  - 大大增加了指令译码难度，延长了指令译码时间，降低了指令执行速度
  - 编译程序难以用优化措施生成高效的目标代码程序，系统执行效率低

6.4.4 CISC与RISC特点

❖2、RISC的特点

❖①指令系统

- 多为使用频率较高的简单指令；
- 指令长度固定，且操作码位置固定
- 指令格式种类少，寻址方式减少且简化
- 只有取数/存数指令访问存储器，其余指令的操作都在寄存器之间进行
- 指令功能简单，故程序长度可能增加

❖②处理器

- 通用寄存器数量相当多，减少了访存次数
- 硬布线控制为主，不用或少用微程序控制
- 采用流水线技术，大部分指令在一个机器周期内完成

❖③性能：

- 可以充分利用VLSI芯片面积
- 使用流水线、超流[illegible]术，实现指令级并行操作[illegible]能
- 易于[illegible]更有效地[illegible]持[illegible]程序

执行每条指令的平均时钟周期数

程序所含的机器指令条数

时钟周期时间

执行程序所需的时间=I*CPI*T_C

- ❖ RISC的I：是CISC的1.2~1.4倍
- ❖ RISC的CPI：≤1，CISC的CPI超过3
- ❖ RISC的T_C：大大下降

❖ 综合性能：RISC优于CISC约2~5倍

CISC与RISC指令系统的对比

对比项目	CISC	RISC
指令条数	几百条	小于100条
寻址方式	丰富、复杂	种类少、简单
指令格式	种类多、可变指令字长、指令字段变化多	种类少、固定指令字长、指令字段位置固定
程序所需指令数	少，机器语言程序短	多，机器语言程序长
通用寄存器数目	少，几个	多，十几个到几十个
实现指令的控制器类型	仍需要微程序控制器实现复杂指令	多采用硬布线控制器
是否利于流水线技术	不利于	利于

- RISC处理器通过更合理的微架构，在性能上超越了传统的Intel的CISC
- 改进：Intel设计了Pentium Pro处理器（P6微架构）
  - x86指令集先被解码为类似于RISC指令的微操作，然后用RISC内核来执行
  - 解决了处理器兼容性问题与CISC架构性能提升问题的矛盾



6.5 指令系统举例

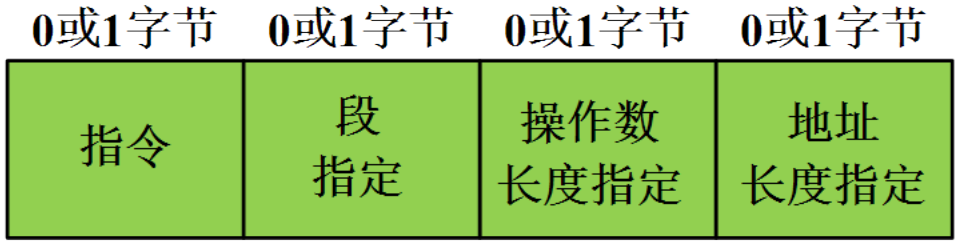


6.5.1 x86 指令系统

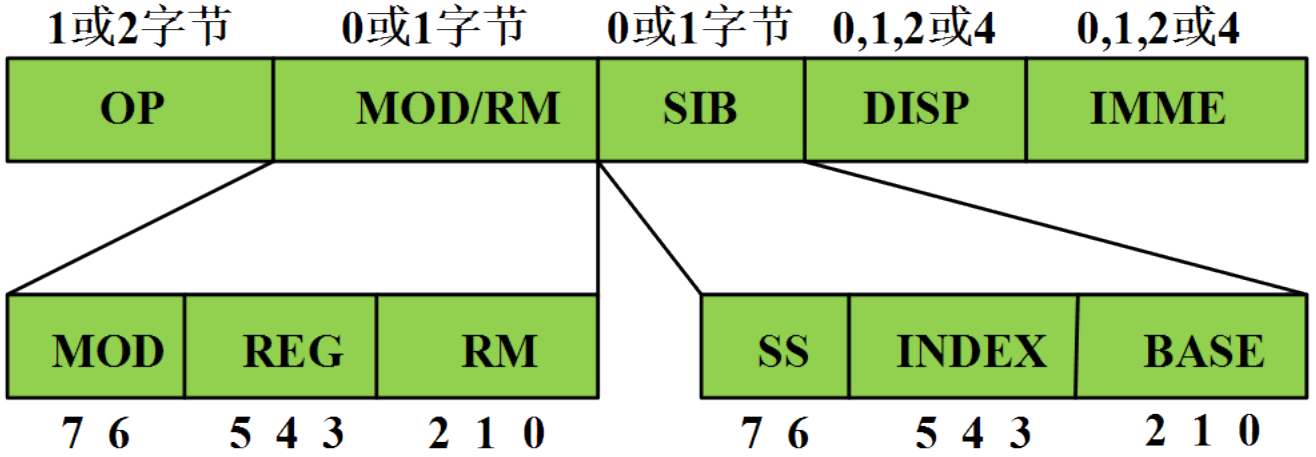
❖ 1、指令类型

- （1）数据传送指令
- （2）算术逻辑操作指令
- （3）转移控制指令
- （4）串操作指令
- （5）处理机控制指令
- （6）特权处理指令

(a)前缀



(b)指令



❖ 2、x86 指令格式

❖ 指令字长可变，8086~Core2（含Pentium系列）从1字节到15字节不等

❖ 两部分组成：指令前缀（可选）+指令本身

6.5.1 x86 指令系统

- 前缀共有4种：
  - （1）LOCK、REP、REPE和REPN
  - （2）段指定前缀：DS/ES/SS/CS
  - （3）操作数长度前缀66H
  - （4）地址长度前缀67H
- 指令包括以下字段：
  - OP：定义指令功能和类型
  - MOD/RM：定义寻址方式
  - SIB：比例变址寻址字节
  - DISP：偏移量
  - IMME：立即数

- 3、x86寻址方式
- 数据寻址方式：8种
- 指令寻址方式：4种

- 逻辑地址：段地址和偏移量构成
- 查找段表，+偏移量→线性地址
- 查找页表→物理地址

6.5.1 x86 指令系统

3、x86寻址方式

数据寻址方式	范例	源操作数	有效地址EA	目的操作数
寄存器寻址	MOV AX, CX	寄存器CX	--	寄存器AX
立即寻址	MOV AL,21H	常数21H	--	寄存器AL
直接寻址	MOV [1000H], AX	寄存器AX	EA =DISP=1000H	存储器单元DS:1000H
寄存器间接寻址	MOV AH, [BX]	存储器单元DS:0200H	EA =(REG)=(BX)=0200H	寄存器AH
寄存器相对寻址	MOV AH, [BX+4]	存储器单元DS:0204H	EA =(REG)+DISP =(BX)+4=0204H	寄存器AH
基址+变址寻址	MOV [BX+SI], DX	寄存器DX	EA =(RB)+(RX) =(BX)+(SI)=0230H	存储器单元DS:0230H
基址+变址+偏移量寻址	MOVAX, Ary [BX+SI]	存储器单元DS:0270H	EA =(RB)+(RX)+DISP =(BX)+(SI)+0040H=0270H	寄存器AX
比例变址寻址	MOVAX, [EBX+4*ESI]	存储器单元DS:02C0H	EA =(RB)+(RX*S) =(EBX)+(ESI)*4=02C0H	寄存器AX

指令寻址方式	范例	目的转移地址EA
直接转移	JMP L	EA=_CS:_IP，指令中给出转移地址的段寄存器值和偏移量
相对转移	JMP L	EA=(EIP)+DISP
寄存器-间接转移	JMP BX	EA=(BX)
变址-间接转移	JMP Table[SI]	EA=(SI)+DISP，DISP为Table的首地址

6.5.2 Yy-z02模型机指令系统



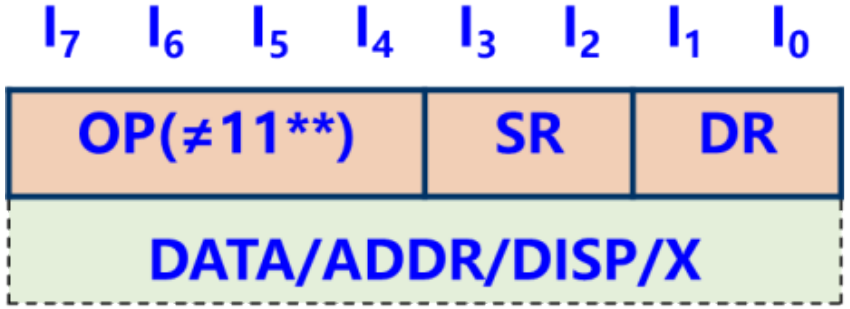
Yy-z02指令格式

Yy-z02寻址方式

Yy-z02指令系统设计

1、Yy-z02指令格式

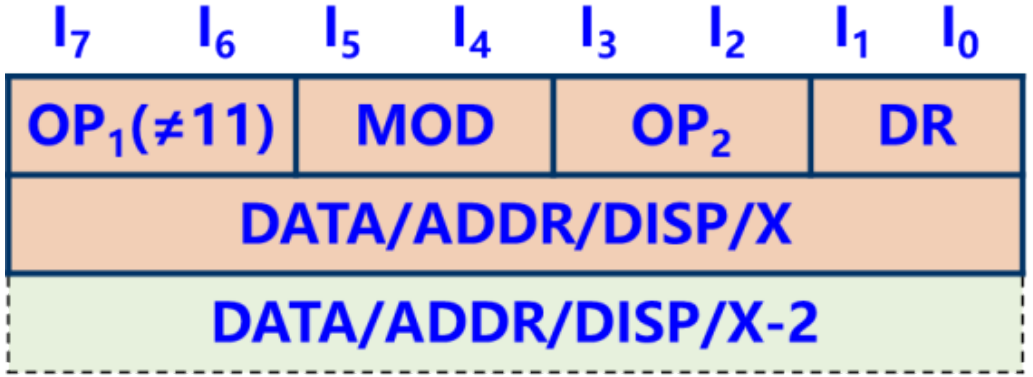
- 格式1：双寄存指令格式
- OP：指令操作码，≠11**，最多12条机器指令



- SR：源寄存器号，2位
  - 4个通用寄存器R_0、R_1、R_2、R_3
- DR：目的寄存器号，2位
- DATA/ADDR/DISP/X：指令的第二个字，可有可无
  - 含义可以自定义，OP指定
  - 立即数　or　直接/间接地址 or 相对偏移量 or 形式地址

1、Yy-z02指令格式

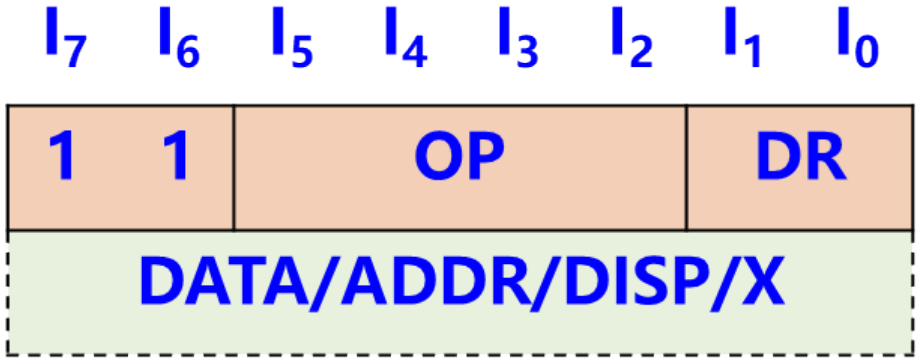
- 格式2：带寻址方式码的指令格式
- OP1：第一指令操作码，2位，格式2特征位
- MOD：寻址方式码，2位，4种寻址方式
  - 寻址方式的定义，可以自行设计，例如：可设计为直接、间接、变址、相对寻址
- OP2：第二指令操作码，2位，格式2指令本身的编码
- DR：目的寄存器



- DATA/ADDR/DISP/X：指令的第二个字
  - 立即数DATA or 直接/间接地址ADDR or 相对寻址的偏移量DISP or 变址寻址的形式地址X
- DATA/ADDR/DISP/X-2：指令第三字，可选，含义由指令操作码和寻址方式码定义

1、Yy-z02指令格式

❖ **格式3：单寄存器的扩展指令格式**



❖OP ： 指令操作码，中间4位，格式3指令的操作码，可容纳16条指令

❖DR：目的寄存器，源操作数 or 目的操作数 or 间址寄存器，均由OP指定

❖DATA/ADDR/DISP/X：指令第二字，可选，其含义仍旧由OP隐含指定的寻址方式来定义

2、Yy-z02寻址方式

- 可实现7种基本寻址方式：寄存器直接、寄存器间接、直接、间接、相对、变址、立即数。
- 相对复杂的寻址方式：直接、间接、相对、变址，可以由MOD字段来定义
- 简单的寻址方式：可以直接由指令操作码OP指定
  - 任何一种寻址方式，均可以直接由指令操作码隐含指定
- 可自行设计一些特殊的寻址方式
  - 相对SR的偏移量寻址：EA=（SR）+ADDR



- 对于格式2，MOD自定义，示例：
  - MOD=00：直接寻址
    - EA=ADDR
    - 操作数=（ADDR)
  - MOD=01：间接寻址
    - EA=(ADDR)
    - 操作数=((ADDR))
  - MOD=10：变址寻址
    - EA=(SI)+X　SI为变址寄存器R_2
    - 操作数=((SI)+X)
  - MOD=11：相对寻址
    - EA=(PC)+DISP
    - 操作数=((PC)+DISP)

3、Yy-z02指令系统设计

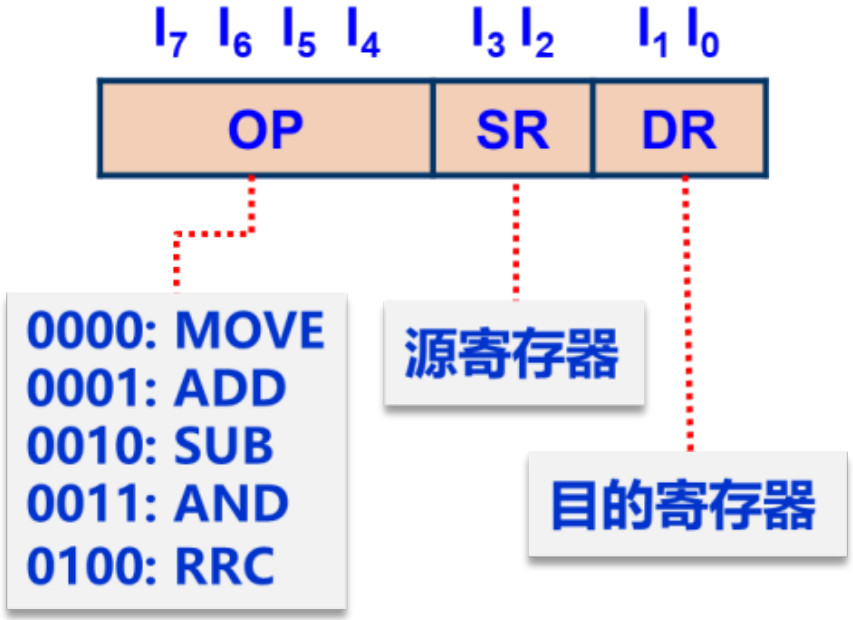
（1）指令设计原则

- 指令的格式必须按照规定的格式设计，即指令第一字有三种格式
  - OP、SR、DR、OP1和OP2、MOD：必须按格式规定固定长度和位置
- 寻址方式的设计，可以根据需要，或由MOD字段定义，或由操作码隐含指定
- 指令操作码的分配设计，要注意规整性

（2）设计举例：16条指令

- 5条双寄存器运算指令
- 4条访存与转移指令
- 3条单目运算指令指令
- 2条I/O指令
- 2条过程控制类指令

（a） 格式1：5条双寄存器运算指令

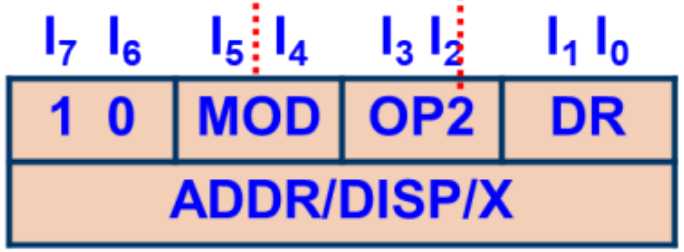


3、Yy-z02指令系统设计

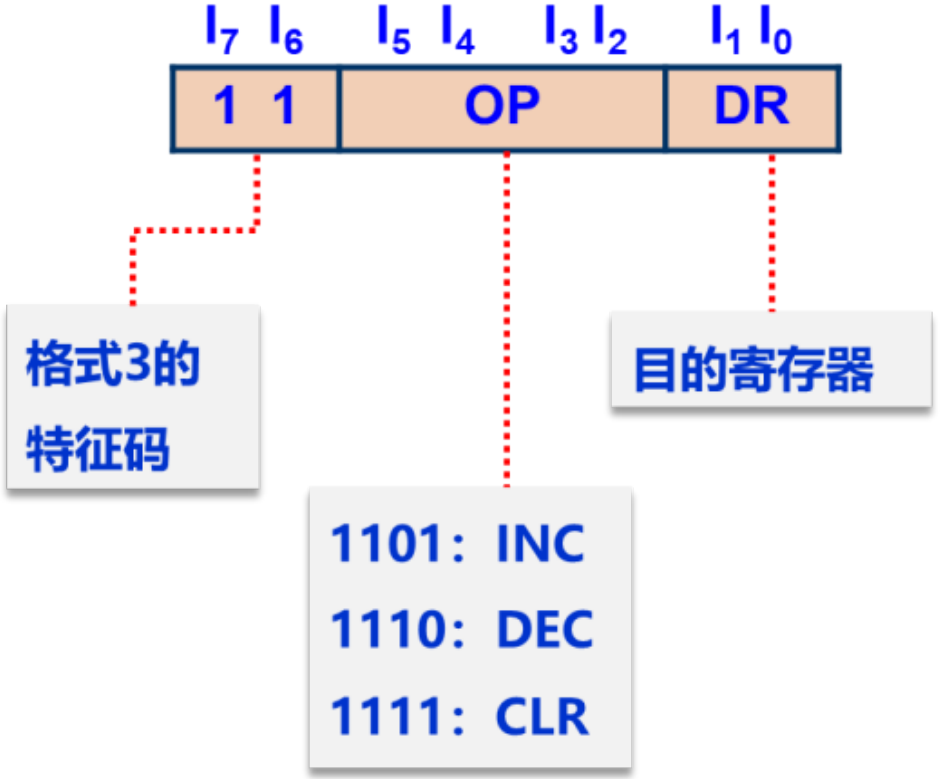
(b) 格式2：4条访存与转移指令

00: 直接寻址，EA=ADDR
01: 间接寻址，EA=(ADDR)
10: 变址寻址，EA=(SI)+X
11: 相对寻址，EA=(PC)+DISP

00: LDA
01: STA
10: JMP
11: JZC

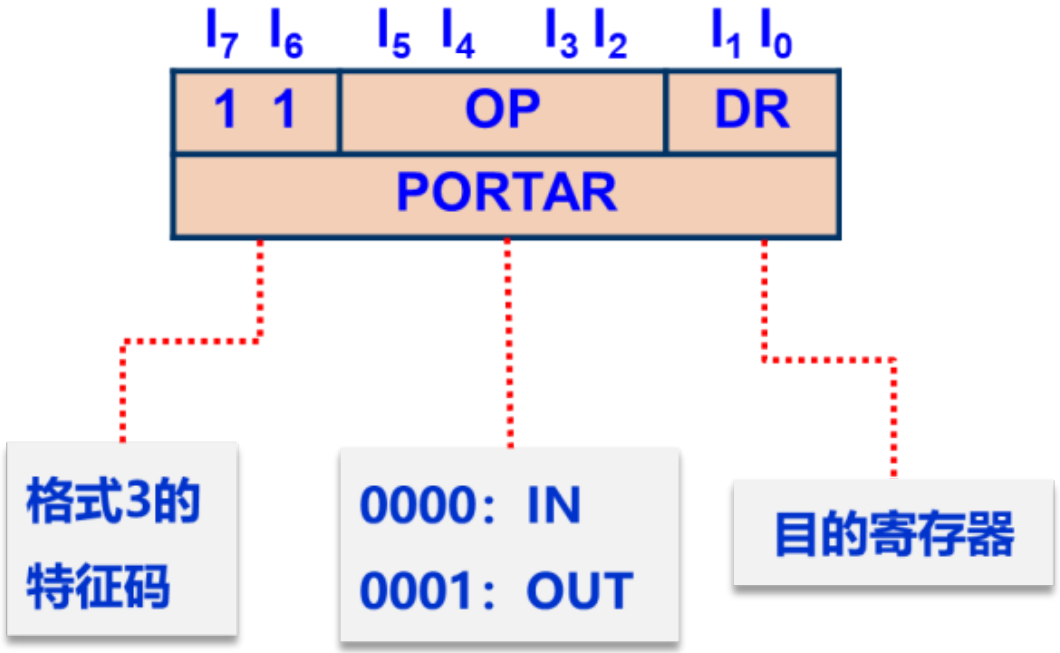


(c) 格式3：3条单目运算指令

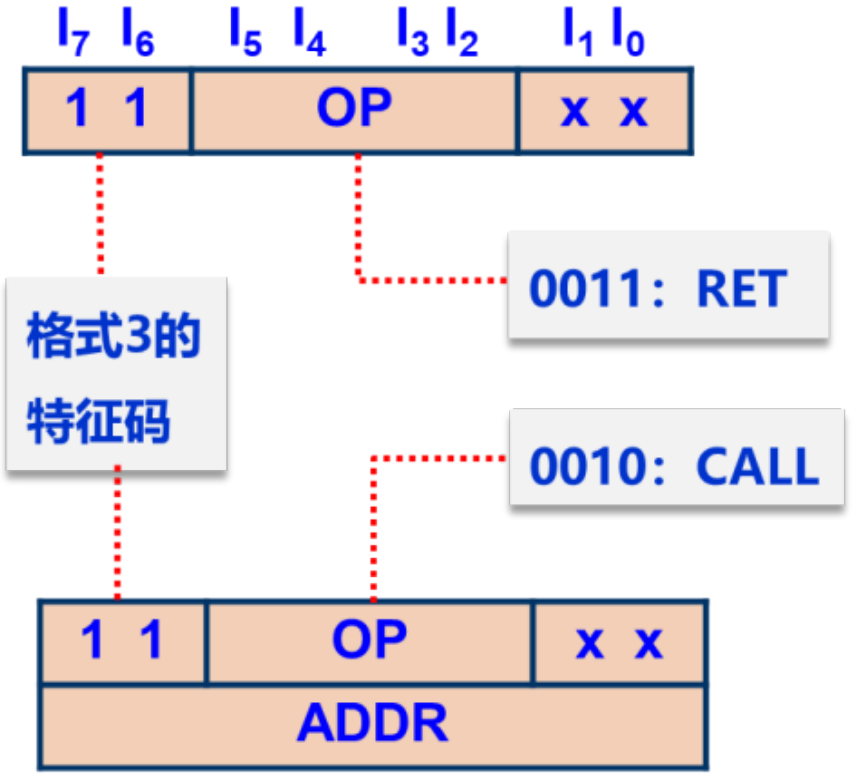


3、Yy-z02指令系统设计

（d） 格式3：2条I/O指令



（e） 格式3：2条过程控制指令



3、Yy-z02指令系统设计

汇编指令	功能	指令格式	指令字长	操作码	寻址方式(源/目的)	指令类型
MOV DR,SR	(SR)→DR	格式1	单字	OP=0000	寄存器/寄存器	传送
ADD DR,SR	(SR)+(DR)→DR	格式1	单字	OP=0001	寄存器/寄存器	算术运算
SUB DR,SR	(DR)-(SR)→DR	格式1	单字	OP=0010	寄存器/寄存器	算术运算
AND DR,SR	(SR)&(DR)→DR	格式1	单字	OP=0011	寄存器/寄存器	逻辑运算
RRC DR,SR	{CF,(SR)}循环右移1位→DR	格式1	单字	OP=0100	寄存器/寄存器	循环移位
LDA DR,[EA]	Mem[EA]→DR	格式2	双字	OP_2=00	MOD/寄存器	取数(访存)
STA DR,[EA]	DR→Mem[EA]	格式2	双字	OP_2=01	寄存器/MOD	存数(访存)
JMP EA	EA→PC	格式2	双字	OP_2=10	MOD	无条件转移
JZC EA	if(FZ+FC=1) EA→PC	格式2	双字	OP_2=11	MOD	条件转移
INC DR	(DR)+1→DR	格式3	单字	OP=1101	寄存器	单目运算
DEC DR	(DR)-1→DR	格式3	单字	OP=1110	寄存器	单目运算
CLR DR	0→DR	格式3	单字	OP=1111	寄存器	单目运算
IN DR,PORTAR	IO[PORTAR]→DR	格式3	双字	OP=0000	直接/寄存器	I/O输入
OUT DR,PORTAR	(DR)→IO[PORTAR]	格式3	双字	OP=0001	寄存器/直接	I/O输出
CALL ADDR	(SP)-1→SP,(PC)→Mem[SP],ADDR→PC	格式3	双字	OP=0010	直接转移，堆栈寻址	子程序调用
RET	Mem[SP]→PC,(SP)+1→SP	格式3	单字	OP=0011	堆栈寻址	子程序返回

6.6 RISC-V指令系统

- 6.6.1 RISC-V指令集概述
- 6.6.2 RV32I的通用寄存器堆
- 6.6.3 RV32I指令格式
- 6.6.4 RV32I的指令类型与编码
- 6.6.5 RV32I的寻址方式



6.6.1 RISC-V指令集概述　1、概况

- **特点：全新的、具有典型RISC特征的指令集架构**
- **起源：2010年加州大学伯克利分校发明**
  - “RISC”：精简指令集计算机
  - “V”：从RISC-I开始设计的第五代指令集
- **RISC-V架构的设计目标**
  - **①完全免费开放：标准手册与架构文档共享在网站（www.riscv.org）上**
  - **②优越的性能：**
    - **能够满足从微控制器到超级计算机等各种复杂程度的处理器需求**
    - **支持从FPGA、ASIC乃至未来器件等多种实现方式，能够高效实现各种微结构，支持定制与加速功能**
    - **兼容目前各种现有软件及编程语言**

6.6.1 RISC-V指令集概述　2、发展现状

- RISC-V基金会：2015年成立
- 成员：全球2000多个著名企业和大学、研究机构
  - 高通、英伟达、三星、IBM、谷歌、华为、Intel等
- 产业生态：进入快速发展期
- 软件工具：GNU编译器（GCC工具链、GDB调试器）、LLVM工具链、OVPsim仿真器、Spike仿真器和QEMU模拟器等
- 操作系统：FreeRTOS、SylixOS、RT-Thread、Linux
- RISC-V：第三个主流CPU架构
  - PC端和服务器市场：x86架构
  - 移动端市场：ARM架构
- RISC-V芯片：2013年1月成功流片
  - 目前芯片出货量已突破百亿颗
  - 2025年有望突破800亿颗
- RISC-V产品：已覆盖微控制器、智能终端、服务器，甚至PC桌面

6.6.1 RISC-V指令集概述　2、发展现状：国内

- 2015年：RISC-V基金会成立时，235个会员中包含20家中国机构
- 2018年：国内成立了两个组织
  - RISC-V产业联盟(China RISC-V Industry Consortium，CRVIC)：中国上海张江
  - 中国开放指令生态RISC-V联盟(China RISC-V Alliance，CRVA)：中科院计算所
  - 有200多家企业、科研院所和高校加入两个组织，共同推动RISC-V生态在中国的快速发展，以期建立中国国产自主、可控、安全的RISC-V平台，促进形成RISC-V产业生态链
- 国内有100+个厂家在研制、生产RISC-V处理器及应用产品
- 阿里平头哥：RISC-V架构的玄铁910芯片，在国内的出货量已超30亿颗
- “国产芯”有望摆脱掣肘，迎来自主可控发展的新阶段

3、RISC-V指令集特点

- 采用模块化的指令子集，每个模块用字母命名
- 任何一个RISC-V指令集架构：必须实现的I模块+可选的扩展指令集模块
- RV32G：32位通用指令集/标准扩展指令集，RV32I+MAFD=RV32IMAFD

指令集模块符号	指令条数	指令集
I	47/59/71	整数指令，基础指令集（RV32I/RV64I/RV128I）
M	8	乘法和除法指令
A	11	存储器原子操作指令和Load-Reserved/store-Conditional指令
F	26	单精度浮点指令
D	26	双精度浮点指令（基于F）
C	46	压缩指令，指令字长为16位

6.6.1 RISC-V指令集概述　3、RISC-V指令集特点

❖ 四个基础整数指令集　❖ 最基础：RV32I、RV64I，本课程以RV32I为例

基础整数指令集符号	RV32I	RV32E	RV64I	RV128I
描述	32位整数指令集	32位嵌入式架构指令集	64位整数指令集，含部分32位指令	128位整数指令集，含部分64位和32位指令
机器字长	32位	32位	64位	128位
地址空间=2^{XLEN}	2^{32}	2^{32}	2^{64}	2^{128}
指令字长ILEN	32位	32位	32位	32位
整数寄存器数量	32个	16个	32个	32个
整数寄存器字长XLEN	32位	32位	64位	128位
指令条数	47	47	59	71

❖ RV32EC：RV32E(16个寄存器)+C(压缩指令,指令字长16位)，小面积、低功耗的嵌入式系统

❖ 指令字长均固定32位；整数采用补码表示；存储器按字节编址，允许小端/大端/大小端交替模式，但指令编码以小端模式存储；本教材默认全部采用小端模式

6.6.2 RV32I的通用寄存器堆

应用程序二进制接口
Application Binary Interface

- 整数寄存器：32×32位，x0~x31，x0=全零
- 浮点寄存器：32×32位，f0~f31
- ABI：规定了每个寄存器的使用约定、寄存器别名

寄存器	ABI别名	用途描述	是否需要在被调用者（子程序）中保存与恢复
x0	zero	零寄存器，硬件置全零	—
x1	ra	返回地址寄存器	是
x2	sp	堆栈指针	是
x3	gp	全局指针	是
x4	tp	线程指针	是
x5	t0	临时寄存器/备用链接寄存器	否
x6~x7	t1~t2	临时寄存器	否
x8	s0/fp	保存寄存器/帧指针	是
x9	s1	保存寄存器	是

寄存器	ABI别名	用途描述	是否需要在被调用者（子程序）中保存与恢复
x10~x11	a0~a1	函数参数/返回值寄存器	否
x12~x17	a2~a7	函数参数寄存器	否
x18~x27	s2~s11	保存寄存器	是
x28~x31	t3~t6	临时寄存器	否
f0~f7	ft0~ft7	浮点临时寄存器	否
f8~f9	fs0~fs1	浮点保存寄存器	是
f10~f11	fa0~fa1	浮点函数参数/返回值寄存器	否
f12~f17	fa2~fa7	浮点函数参数寄存器	否
f18~f27	fs2~fs11	浮点数保存寄存器	是
f28~f31	ft8~t11	浮点数临时寄存器	否

6.6.2 RV32I的通用寄存器堆

① 零寄存器zero：x0，始终读出全零，写入无效

② 子程序调用相关的寄存器：

- 返回地址寄存器ra：x1
  - 执行子程序调用指令：PC+4→ra
  - 执行子程序返回指令：ra→PC，返回断点
- 函数参数寄存器a0~a7：x10~x17共计8个
  - 保存函数入口参数
  - 如果参数超出8个，则需要通过堆栈传递
- 函数返回值寄存器a0~a1：x10~x11共计2个
  - 传递函数返回值

③ 保存寄存器s0~s11：x8~x9、x18~x27共计12个

- 被子程序使用时，必须“掩盖使用痕迹”，即要保证其值在子程序执行前后保持不变

④ 临时寄存器t0~t6：x5~x7、x28~x31共计7个，可被子程序任意使用，无需保存与恢复

⑤ 堆栈指针sp：x2，保存堆栈栈顶的内存地址

❖ 控制与状态寄存器CSR：4096×32位，配置/记录处理器硬件的运行状态，用12位地址访问



6.6.3 RV32I指令格式

- 指令字长：固定为32位，必须在4字节的边界上对齐
- 指令格式：6种

功能码　源寄存器　目的寄存器　操作码

指令格式类型	31 25	24 20	19 15	14 12	11 7	6 0	指令类型
	7位	5位	5位	3位	5位	7位	
R型	funct7	rs2	rs1	funct3	rd	opcode	运算指令
I型	imm[11:0]		rs1	funct3	rd	opcode	装数/立即数运算
S型	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	存数指令
B型	imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode	条件分支指令
U型	imm[31:12]				rd	opcode	大立即数
J型	imm[20,10:1,11,19:12]				rd	opcode	无条件跳转指令

立即数

6.6.3 RV32I指令格式

- 操作码opcode：7位，固定在指令字的最低7位，指定指令的格式类型。
- 寄存器字段：3个、5位，源寄存器rs1/rs2、目的寄存器rd，固定在相同位置
- 立即数字段：12位、20位两种，立即数最高位=指令码最高位inst[31]
- 各种格式：用于不同指令类型
  - R型格式：运算指令，三个操作数均是寄存器类型
  - I型格式：立即数运算操作指令和装数指令，源操作数由寄存器rs1和12位的立即数字段指定，有目的寄存器rd
  - S型指令：存数指令，没有rd，有操作数rs1、rs2与2个立即数字段（共12位）
  - B型格式：分支指令，12位立即数字段指出跳转的偏移量（S型的变体）
  - U型格式：大立即数指令，20位立即数字段
  - J型格式：无条件跳转指令，20位立即数字段（U型的变体）

6.6.3 RV32I指令格式

- 5种指令格式包含立即数字段imm：12位和20位立即数
- imm→立即数imm[31:0] 方法
  - 零扩展：高位添0（CSR指令、I型移位指令）
  - 符号扩展：高位添符号位inst[31]（其他指令）

- inst[y]：指令码的第y位
- n{x}：x复制n次
- 20{inst[31]}：inst[31]复制20次，即符号扩展

I1_imm[31:0]	31																								5	4					0
I型移位指令立即数	27’ b0																									inst[24:20]					
I2_imm[31:0]	31																			12	11										0
I型其他指令立即数	20{inst[31]}																				inst[31:20]										
S_imm[31:0]	31																			12	11					5	4				0
S型指令立即数	20{inst[31]}																				inst[31:25]						inst[11:7]				
B_imm[31:0]	31																			12	11				10	5	4		1	0	
B型指令立即数	20{inst[31]}																				inst[7]				inst[30:25]		inst[11:8]			0	
U_imm[31:0]	31																			12	11										0
U型指令立即数	inst[31:12]																				12{0}										
J_imm[31:0]	31																	20	19	12	11				10				1	0	
J型指令立即数	12{inst[31]}																		inst[19:12]		inst[20]				inst[30:21]					0	

6.6.4 RV32I的指令类型与编码

- 1、整数运算类指令
- 2、U型数据处理指令
- 3、访存指令
- 4、转移控制指令
- 5、存储器屏障指令
- 6、系统指令



1、整数运算类指令 ——寄存器运算指令10条（R型）

❖ R型运算指令：opcode=0110011

❖ 由fuct3、funct7区分指令

R型整数运算指令							
位数	7位	5位	5位	3位	5位	7位	指令功能
字段	funct7	rs2	rs1	funct3	rd	opcode	
add rd,rs1,rs2	0000000	rs2	rs1	000	rd	0110011	rs1+rs2→rd（加法）
sub rd,rs1,rs2	0100000	rs2	rs1	000	rd	0110011	rs1-rs2→rd（减法）
sll rd,rs1,rs2	0000000	rs2	rs1	001	rd	0110011	rs1<<rs2→rd（逻辑左移，低位补0）
slt rd,rs1,rs2	0000000	rs2	rs1	010	rd	0110011	if(rs1<rs2) 1→rd else 0→rd(有符号数比较小于置数)
sltu rd,rs1,rs2	0000000	rs2	rs1	011	rd	0110011	if(rs1<rs2) 1→rd else 0→rd(无符号数比较小于置数)
xor rd,rs1,rs2	0000000	rs2	rs1	100	rd	0110011	rs1⊕rs2→rd（异或）
srl rd,rs1,rs2	0000000	rs2	rs1	101	rd	0110011	rs1>>rs2→rd（逻辑右移，高位补0）
sra rd,rs1,rs2	0100000	rs2	rs1	101	rd	0110011	rs1>>>rs2→rd（算术右移，高位补符号位）
or rd,rs1,rs2	0000000	rs2	rs1	110	rd	0110011	rs1 | rs2→rd（或运算）
and rd,rs1,rs2	0000000	rs2	rs1	111	rd	0110011	rs1 & rs2→rd（与运算）

1、整数运算类指令 ——立即数运算指令9条（I型）

Why?

❖ I型运算指令：opcode=0010011

❖ 由funct3区分指令；没有减法指令

I型立即数运算指令							
位数	12位		5位	3位	5位	7位	指令功能
字段	imm[11:0]		rs1	funct3	rd	opcode	
addi rd,rs1,imm12	imm[11:0]		rs1	000	rd	0010011	rs1+SE32(imm12)→rd（立即数加法）
slti rd,rs1,imm12	imm[11:0]		rs1	010	rd	0010011	if(rs1<SE32(imm12)) 1→rd else 0→rd(有符号数比较小于置数)
sltiu rd,rs1,imm12	imm[11:0]		rs1	011	rd	0010011	if(rs1<SE32(imm12)) 1→rd else 0→rd(无符号数比较小于置数)
xori rd,rs1,imm12	imm[11:0]		rs1	100	rd	0010011	rs1⊕SE32(imm12)→rd(立即数异或运算)
ori rd,rs1,imm12	imm[11:0]		rs1	110	rd	0010011	rs1 | SE32(imm12)→rd（立即数或运算）
andi rd,rs1,imm12	imm[11:0]		rs1	111	rd	0010011	rs1 & SE32(imm12)→rd（立即数与运算）
slli rd,rs1,shamt	0000000	shamt	rs1	001	rd	0010011	rs1<<shamt→rd（逻辑左移）
srli rd,rs1,shamt	0000000	shamt	rs1	101	rd	0010011	rs1>> shamt→rd（逻辑右移）
srai rd,rs1,shamt	0100000	shamt	rs1	101	rd	0010011	rs1>>> shamt→rd(算术右移)

❖ 除了移位指令的shamt字段，立即数均使用符号扩展，包括逻辑运算指令、无符号数指令sltiu

1、整数运算类指令 ——总结

- 19条基本的运算指令：对称地分为R型和I型两种格式
  - 3条算术运算指令（加法和减法）、6条逻辑运算指令（与、或、异或）
  - 6条移位指令（逻辑左移、逻辑右移和算术右移）
  - 4条比较小于置数指令（带符号数slt、slti，不带符号数sltu、sltiu）
  - R型：所有操作数均为寄存器类型，功能是 rs1 op rs2 → rd
  - I型：第二操作数为立即数，功能是 rs1 op SE（imm12） → rd
    - 没有subi rd,rs1,imm12指令，可以用addi rd,rs1,-imm12代替



2、U型数据处理指令 ——大立即数指令2条（U型）

- U型大立即数指令：由opcode直接区分指令
  - lui：将20位的立即数imm20装入rd高位，rd的低12位为0；{imm20,12{0}}→rd
  - auipc：将20位的立即数imm20作为高位、低12位清零构成一个32位偏移量，与PC相加，结果装入rd。PC+{imm20,12{0}}→rd

U型大立即数传送指令				
位数	20位	5位	7位	指令功能
字段	imm[31:12]	rd	opcode	
lui rd,imm20	imm[31:12]	rd	0110111	{imm20,12{0}}→rd（装立即数高20位）
auipc rd,imm20	imm[31:12]	rd	0010111	PC+{imm20,12{0}}→rd（加立即数到PC并取回）

- 例：lui x5,0x123
- 结果x5=?
- x5=0x 0012 3000

例题

❖ 【例6.5】一个RISC-V的32位计算机的C语言片段如下：

```
int a,b,c,w;
a = -3;        b = 1000;        c = 7000;    w=8a+b-c;
```

假设C编译器给a、b、c、w四个变量分配的寄存器分别为x5~x8，请写出上面4条C语言语句对应的RV32I的汇编程序段及其指令代码。

① a = -3：a分配了寄存器x5

- 用立即数加法指令实现：addi x5,x0,-3　　#x0+(-3)→x5

② b = 1000：　b分配了寄存器x6

- 用立即数加法指令实现：　addi x6,x0,1000 #x0+1000→x6

❖ 执行时：立即数 ffd h符号扩展成32位=ffff fffd

$h=[-3]_{补}$

格式	imm[11:0]	rs1	funct3	rd	opcode
addi x5,x0,-3字段	-3	x0	add	x5	IMM_OP
addi x5,x0,-3编码	1111 1111 1101	00000	000	00101	0010011
addi x6,x0,1000字段	1000	x0	add	x6	IMM_OP
addi x6,x0,1000编码	0011 1110 1000	00000	000	00110	0010011

$[-3]_{补}$ = ffd h

$[1000]_{补}$ = 3e8 h

例题

③ c = 7000=1b58 h：13位二进制，立即数字段的12位不能表示

- lui指令：将高20位00001h装入寄存器x7
- addi指令：加x7将其低12位b58h装入
- 执行addi指令：SE32（b58h）=ffff fb58h， ffff fb58h +0000 1000h=0000 0b58h ≠0000 1b58 h=7000，未达到预期。原因是： b58h最高位为1，视为负数。

```
lui x7, 1                #0000 1000h→x7
addi x7,x7,b58h          #x7+b58h→x7
```

- 如何得到正确的大立即数?
- 编译器：lui的立即数=立即数的32位补码imm32的高20位+第11位（即imm32[31:12]+imm32[11]）

- 这里：imm32[31:12]=00001h，imm32[11]=1，故两条指令为：
- lui x7, 2　　#0000 2000h→x7
- addi x7,x7,b58h　#x7+b58h→x7

格式	imm[31:12]	rd	opcode
lui x7, 2字段	2	x7	LUI
lui x7, 2编码	0000 0000 0000 0000 0010	00111	0110111

- ❖ lui指令编码：0000 23b7h
- ❖ addi指令编码：b583 8393h

例题

④ w=8a+b-c：

- 8a：左移指令slli指令，将a左移3位
  - 逻辑左移指令在保证左移结果（有符号数）不溢出的情况下，可以当做算术左移指令；
  - 左移运算比乘法运算更快
- +b-c：执行加法和减法

```
slli x8,x5,3        #x5 << 3→x8
add x8,x8,x6        #x8+x6→x8
sub x8,x8,x7        #x8-x7→x8
```

指令/机器码	funct7	rs2/shamt	rs1	funct3	rd	opcode
slli x8,x5,3	0000000	3	x5	001	x8	IMM_OP
0032 9413h	0000000	00011	00101	001	01000	0010011
add x8,x8,x6	0000000	x6	x8	000	x8	OP
0064 0433h	0000000	00110	01000	000	01000	0110011
sub x8,x8,x7	0100000	x7	x8	000	x8	OP
40740433h	0100000	00111	01000	000	01000	0110011

3、访存指令 ——装数指令5条（I型）

- I型取数指令：opcode=0000011
- 由funct3区分指令

I型装数指令						
位数	12位	5位	3位	5位	7位	指令功能
字段	imm[11:0]	rs1	funct3	rd	opcode	
lb rd, imm12(rs1)	imm[11:0]	rs1	000	rd	0000011	SE32(Mem8[rs1+SE32(imm12)])→rd (取字节数据)
lh rd, imm12(rs1)	imm[11:0]	rs1	001	rd	0000011	SE32(Mem16[rs1+SE32(imm12)])→rd (取半字数据)
lw rd, imm12(rs1)	imm[11:0]	rs1	010	rd	0000011	Mem32[rs1+SE32(imm12)]→rd (取字数据)
lbu rd, imm12(rs1)	imm[11:0]	rs1	100	rd	0000011	UE32(Mem8[rs1+SE32(imm12)])→rd (取无符号的字节数据)
lhu rd, imm12(rs1)	imm[11:0]	rs1	101	rd	0000011	UE32(Mem16[rs1+SE32(imm12)])→rd (取无符号的半字数据)

- 装数指令的功能：Mem[EA]→rd
- EA=rs1+SE32（imm12）
- 取出的字节、半字数据→32位?
  - SE32/UE32

Unigned Extend to 32 bits

- Mem8：地址单元的字节数据
- Mem16：地址单元的半字数据

3、访存指令 ——存数指令3条（S型）

- I型存数指令：opcode=0100011
- 由funct3区分指令

S型存数指令							
位数	7位	5位	5位	3位	5位	7位	
字段	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	指令功能
sb rs2, imm12(rs1)	imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	Reg8[rs2]→Mem8[rs1+SE32(imm12)]（存字节数据）
sh rs2, imm12(rs1)	imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	Reg16[rs2]→Mem16[rs1+SE32(imm12)]（存半字数据）
sw rs2, imm12(rs1)	imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	rs2→Mem32[rs1+SE32(imm12)]（存字数据）

- sb： store byte
- sh： store half word
- sw：store word

- 存数指令的功能：rs2→Mem[EA]
- EA=rs1+SE32（imm12）
  - imm12：两个字段拼接
- 写入数据尺寸：字节sb、半字sh、字sw

- Reg8：寄存器低8位
- Reg16：寄存器低16位

3、访存指令 ——举例

❖ 【例6.6】 假设某RV32I架构计算机，地址401280H的主存单元数据为234589ABH，请写出下面程序段每条指令执行后的结果。

```
lui     t0,0x401
addi    t0,t0,0x200
lw      t1,0x80(t0)
lh      t2,0x80(t0)
lhu     t3,0x80(t0)
lh      t4,0x82(t0)
```

```
lb      t5,0x80(t0)
lb      t6,0x83(t0)
sw      t4,0x80(t0)
sh      t3,0x82(t0)
sb      t5,0x83(t0)
```

❖ （1）前两条指令：

```
lui     t0,0x401        #t0=0x401000
addi    t0,t0,0x200     #t0=0x401200
```

3、访存指令 ——举例

❖ （2）一组取数指令：内存的数据存储顺序？小端模式

字地址 401280H	23H	45H	89H	ABH
字节地址	401283H	401282H	401281H	401280H

- ①根据EA从主存取出规定尺寸的数据；②进行符号扩展 / 零扩展；③存入目的寄存器

t0=0x401200

```
lw      t1,0x80(t0)      #t1=Mem32[401280H]=234589ABH
lh      t2,0x80(t0)      #t2=SE32 (Mem16[401280H]) =FFFF89ABH
lhu     t3,0x80(t0)      #t3=UE32 (Mem16[401280H]) =000089ABH
lh      t4,0x82(t0)      #t4=SE32 (Mem16[401282H]) =00002345H
lb      t5,0x80(t0)      #t5=SE32 (Mem8[401280H]) =FFFFFFABH
lb      t6,0x83(t0)      #t6=SE32 (Mem8[401283H]) =00000023H
```

3、访存指令 ——举例

❖ （3）最后一组存数指令：

字地址 401280H

23H	45H	89H	ABH
401283H	401282H	401281H	401280H

字节地址

- ①按照规定尺寸，取出rs2对应寄存器的低位数据；②写入EA指定的存储器单元。

t1=234589ABH
t2=FFFF89ABH
t3=000089ABH

t4=00002345H
t5=FFFFFFABH
t6=00000023H

sw	t4,0x80(t0)	#Mem32[401280H]	=t4	=00002345H
sh	t3,0x82(t0)	#Mem16[401282H]	=Reg16[t3]	=89ABH
sb	t5,0x83(t0)	#Mem8[401283H]	=Reg8[t5]	=ABH

❖ 执行完三条存数指令后，主存单元401280H的内容是?

ABAB 2345H



4、转移控制指令

J型无条件跳转指令：jal			
位数	20位	5位	7位
字段	imm[20,10:1,11,19:12]	rd	opcode
jal rd,offset20	offset20（=offset[20:1])	rd	1101111

指令功能
PC+4→rd,PC+SE32(offset20*2)→PC（相对跳转并链接）

I型无条件跳转指令：jalr					
位数	12位	5位	3位	5位	7位
字段	imm[11:0]	rs1	funct3	rd	opcode
jalr rd,offset12(rs1)	offset12	rs1	000	rd	1100111

指令功能
PC+4→rd,rs1+SE32(offset12)→PC（寄存器相对跳转并链接）

B型条件分支指令							
位数	7位	5位	5位	3位	5位	7位	
字段	imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode	指令功能
beq rs1,rs2,offset12	offset7	rs2	rs1	000	offset5	1100011	if(rs1=rs2)PC+SE32(offset12*2)→PC（相等则跳转）
bne rs1,rs2,offset12	offset7	rs2	rs1	001	offset5	1100011	if(rs1≠rs2)PC+SE32(offset12*2)→PC（不相等则跳转）
blt rs1,rs2,offset12	offset7	rs2	rs1	100	offset5	1100011	if(rs1＜rs2)PC+SE32(offset12*2)→PC（小于则跳转）
bge rs1,rs2,offset12	offset7	rs2	rs1	101	offset5	1100011	if(rs1≥rs2)PC+SE32(offset12*2)→PC（大于等于则跳转）
bltu rs1,rs2,offset12	offset7	rs2	rs1	110	offset5	1100011	if(rs1＜rs2)PC+SE32(offset12*2)→PC（无符号比较小于则跳转）
bgeu rs1,rs2,offset12	offset7	rs2	rs1	111	offset5	1100011	if(rs1≥rs2)PC+SE32(offset12*2)→PC（无符号比较大于等于则跳转）

4、转移控制指令 ——无条件跳转指令jal/jalr

J型无条件跳转指令：jal				
位数	20位		5位	7位
字段	imm[20,10:1,11,19:12]		rd	opcode
jal rd,offset20	offset20（=offset[20:1]）		rd	1101111

链接　跳转：相对PC

PC+4→rd,
PC+SE32(offset20*2)→PC
(相对跳转并链接)

I型无条件跳转指令：jalr					
位数	12位	5位	3位	5位	7位
字段	imm[11:0]	rs1	funct3	rd	opcode
jalr rd,offset12(rs1)	offset12	rs1	000	rd	1100111

链接　跳转：相对rs1

PC+4→rd,
rs1+SE32(offset12)→PC
(寄存器相对跳转并链接)

- **jal和jalr指令：跳转并链接指令**
  - 保存返回地址：下一条指令的地址（PC+4）保存到rd
  - 跳转：目标地址置入PC

4、转移控制指令 ——无条件跳转指令jal/jalr

❖ jal指令：

- 指令格式：J型
- 寻址方式：相对寻址
- 目标转移地址：PC+SE32（offset20*2），即：偶地址
- offset20含义：当前PC与目标指令之间的半字数
  - imm32={11{offset[20]}, offset[20:1], 0 }
- 跳转范围：PC±2^{19}个半字（即±1MB）的地址区间
  - 准确相对范围：-2^{20}~ 2^{20}-2

不是PC+4

❖ jalr指令：

- 指令格式：I型
- 寻址方式：相对寄存器寻址
- 目标转移地址：rs1+SE32（offset12）
- offset12含义：目标指令与rs1之间的偏移量
- 跳转范围：相对寄存器rs1前后2KB的区间

4、转移控制指令 ——无条件跳转指令jal/jalr

❖jal和jalr指令的功用

①可实现：子程序的调用与返回

call subroutine

```
                jal x1,subroutine      #PC+4→x1,&subroutine→PC
                ……
subroutine:                            #子程序名，代表子程序入口地址（第一条指令地址）
                ……
return          jalr x0,x1,0           #PC+4→x0，x1+0→PC；子程序返回
```

无法写入x0

- ❖ jal x1,subroutine ： 相当于call subroutine
  - x1：返回地址寄存器ra
- ❖ subroutine：子程序名，汇编器计算目标地址与当前指令之间的偏移量，形成机器指令编码
- ❖ jalr x0,0(x1)：指令事实上无法写入零寄存器x0，故只是执行从x1/ra中取出返回地址送PC，即执行了return操作（汇编伪指令为ret）

4、转移控制指令 ——无条件跳转指令jal/jalr 举例

❖ 【例6.7】假设：

❖ jal和jalr的指令编码?

```
地址
0000 0010h:    jal x1,subroutine
               ……
0000 0100h:    ……        #subroutine入口
               ……
               jalr x0,x1,0
```

指令/机器码	imm[20,10:1,11,19:12]	rd	opcode
jal x1,subroutine	00078h=0 0000 0000 0 0001111000	x1	1101111
0f00 00efh	0 0001111000 0 00000000	00001	1101111

❖ ①计算当前jal指令地址（PC值）和subroutine入口地址之间的偏移量：

- offset32 =0000 0100h-0000 0010h = 0000 00f0h = 0000 0000 0000 0000 0000 0000 1111 0000b
- jal指令偏移量是低21位：offset21[20:0]=0 0000 0000 0000 1111 0000b
- J型格式：指令编码中只取20位offset20=offset21[20:1]=00078h（最低位的0舍弃）

❖ ②按照imm[20,10:1,11,19:12]重新组装：0 0001111000 0 00000000

❖ jalr x0,x1,0指令：I型格式

指令/机器码	imm[11:0]	rs1	funct3	rd	opcode
jalr x0,x1,0	0	x1	000	x0	1100111
0000 8067h	0000 0000 0000	00001	000	00000	1100111

4、转移控制指令 ——无条件跳转指令jal/jalr 举例

- 【例6.8】call subroutine是一条伪指令，用于子程序的远调用，即：被调用的子程序（callee）地址与当前调用指令的偏移量超出了调用指令（caller，jal指令）的前后1MB范围。假设子例程subroutine的入口地址是0040 0100h，调用指令call subroutine地址是0010 0000h，则伪指令call subroutine如何实现？请写出相关的指令序列及其指令编码。

- subroutine地址超出了jal指令的可相对转移的范围，直接使用jal指令不能生效。
- 先使用auipc指令：auipc x1, 00300h
  - 计算子例程入口地址与当前地址的偏移量：offset32=00400100h-00100000h=00300100h
  - 取高20位offset32[31:12]+offset[11]= 00300h+0=00300h
  - 拼接12个0，加上PC存入x1/ra寄存器
- 再使用jalr指令进行跳转并链接：jalr x1, 100h(x1)
  - 偏移量取offset32[11:0]
- call subroutine伪指令实际上是以下两条指令实现：

地址	机器指令	汇编指令	操作	结果
00100000h	0030 0093h	auipc x1, 00300h	PC+00300 000h→x1	x1=00400000h
00100004h	1000 80e7h	jalr x1, 100h(x1)	PC+4→x1,x1+100h→PC	x1=00100008h PC=00400100h

4、转移控制指令 ——无条件跳转指令jal/jalr

❖jal和jalr指令的功用

②可实现：无条件跳转指令

❖ 当rd=x0时，返回地址PC+4无法写入x0，此时：

- jal x0,offset20：j offset20，无条件的相对跳转指令
- jalr x0,rs1,offset12：j offset(rs1)，无条件相对寄存器跳转指令；

  特别地：

  伪指令

  - 偏移量为0：跳转到rs1指定的32位绝对地址；
  - rs1为x0：跳转到32位地址空间的最低2KB或者最高2KB。

4、转移控制指令 ——条件分支指令bcc

B型条件分支指令							
位数	7位	5位	5位	3位	5位	7位	
字段	imm[12, 10:5]	rs2	rs1	funct3	imm[4:1, 11]	opcode	指令功能
beq rs1,rs2,offset12	offset7	rs2	rs1	000	offset5	1100011	if(rs1=rs2)PC+SE32(offset12*2)→PC (相等则跳转)
bne rs1,rs2,offset12	offset7	rs2	rs1	001	offset5	1100011	if(rs1≠rs2)PC+SE32(offset12*2)→PC (不相等则跳转)
blt rs1,rs							C
bge rs1,r							C
bltu rs1,r							C
bgeu rs1,r							C

- 指令格式：B型
- 功能：rs1和rs2的数据进行比较，只有结果符合条件时才发生跳转（目标地址→PC）
- 转移寻址方式：相对转移
- 目标地址：PC+SE32(offset12*2)，偶地址，offset12[12:1]
  - imm32={19{offset[12]}, offset[12:1], 0 }
- offset12含义：当前PC与目标指令之间的半字数
- 跳转范围：PC±2^{11}个半字（即±4KB）的地址区间

4、转移控制指令 ——条件分支指令bcc 举例

- 【例6.9】下面一段C语言程序实现了在int类型数组data中搜索首个值等于y的数据：

```
int   data[ LENGTH ];        int i=0,y;
while (data[i]!=y)
        i+=1;
```

- 假设宿主机32位
- 数组data在主存的首地址：s4（x20）
- 变量i：s9（x25）
- 变量y：s10（x26）
- ①写出RISC-V的汇编语言程序
- ②翻译成机器语言程序，存放在主存地址20000单元开始的空间内

4、转移控制指令 ——条件分支指令bcc 举例

```
while (data[i]!=y)
        i+=1;
```

data首地址：s4（x20）

变量i：s9（x25）

变量y：s10（x26）

❖ 解：这是一个while循环程序，结束的条件是数组元素等于y

① 从主存取出数组元素data[i]到临时寄存器：主存地址?

- data[i]的主存地址=data数组的基址+4*i
- 4*i可以用i左移2位实现　取出data[i]的汇编程序段：

```
slli    t1,    s9,    2      #t1= i*4
add     t1,    t1,    s4     #t1 = &data[i]
lw      t2,    0(t1)         #t2 = Mem[t1] = data[i]
```

② 测试是否要执行循环体：如果data[i]=y，则退出循环，跳转到exit；否则执行循环体

i=i+1；这里使用条件分支指令beq完成比较：

```
beq     t2,    s10,   exit   #if(data[i]=y)  goto  exit
addi    s9,    s9,    1      #i=i+1
```

4、转移控制指令 ——条件分支指令bcc 举例

③ 执行到循环体的最后：需要无条件回到循环体入口，定义循环入口标号为Loop，即slli指令

- 这里需要使用无条件跳转指令：

```
jal     x0,Loop     #goto Loop
exit:
```

④ 按照指令格式，逐条编码机器指令，其中beq指令的exit标号的偏移量是+12；而jal指令中Loop标号的偏移量是-20

地址	机器代码	汇编指令				注释
20000	002c9313h	Loop: slli	t1,	s9,	2	t1 = i*4
20004	01430333h	add	t1,	t1,	s4	t1 = &data[i]
20008	00032383h	lw	t2,	0(t1)		t2 = Mem[t1] = data[i]
20012	01a38663h	beq	t2,	s10,	exit	if(data[i]=y) goto exit
20016	001c8c93h	addi	s9,	s9,	1	i=i+1
20020	fedff06fh	jal	x0,	Loop		goto Loop
20024		exit:				退出while循环

5、存储器屏障指令

- Hart：硬件线程（Hardware Thread）
- RISC-V支持**硬件超线程技术**：一个物理的处理器核中实现多份硬件线程，每份线程都有自己独立的寄存器组等上下文资源，但所有硬件线程复用与共享同一个物理CPU核的运算与处理资源
- RISC-V支持存储器的**松散一致性模型**，允许每个单核或者Hart改变其存储器访问指令的执行顺序。从全局来看，**执行结果会变得完全不可确定**
- 解决方法：使用存储器屏障指令fence和fence.i

位数	4位	4位				4位				5位	3位	5位	7位	指令功能
字段	fm	pred				succ				rs1	funct3	rd	opcode	
fence	0000	PI	PO	PR	PW	SI	SO	SR	SW	00000	000	00000	0001111	同步数据访问指令
fence.i	0000 0000 0000									00000	001	00000	0001111	同步指令流和数据流

5、存储器屏障指令

- **fence指令：约束数据访问指令的执行顺序**
  - **栅栏：在fence之前的数据访问指令一定比fence之后的数据访问指令先执行**
  - **访问操作分为4种：：设备输入(I)、设备输出(O)、存储器读取（R）和存储器写入（W）**
  - **pred字段：含PI、PO、PR和PW四位，指定fence之前的处理(predecessor)操作组合**
  - **succ字段：含SI、SO、SR和SW四位，指定fence之后的处理器(successor)操作组合；**
- **fence.i指令：同步指令流和数据流**
  - **栅栏：“在fence.i指令之前所有指令进行的数据访问结果”一定能够被“在fence.i指令之后所有指令进行的取指令操作”访问到**
  - **解决了“先装载程序块到存储器、后执行程序块时，如何保证取到的指令是最新装载的程序块”这个问题**

6、系统指令 ——CSR读写指令

- 访问可能需要特权访问的系统功能
  - 6条CSR读写指令、2条调试/断点指令，均为I-type指令格式
- CSR原子读写指令：访问CSR(控制和状态寄存器)，配置模式、使能位，或者记录程序运行的状态，它们是处理器内部的专用寄存器，使用12位地址进行编码

I型CSR访问指令						
位数	12位	5位	3位	5位	7位	
字段	csr	rs1	funct3	rd	opcode	指令功能
csrrw rd,csr,rs1	csr	rs1	001	rd	1110011	csr→rd, rs1→csr（csr原子读写）
csrrs rd,csr,rs1	csr	rs1	010	rd	1110011	csr→rd, rs1 | csr→csr（csr原子读、置位）
csrrc rd,csr,rs1	csr	rs1	011	rd	1110011	csr→rd, $\overline{rs1}$& csr→csr（csr原子读、清零）
csrrwi rd,csr,uimm	csr	uimm	101	rd	1110011	csr→rd, UE32(uimm)→csr（csr原子读写）
csrrsi rd,csr,uimm	csr	uimm	110	rd	1110011	csr→rd, UE32(uimm) | csr→csr（csr原子读、置位）
csrrci rd,csr,uimm	csr	uimm	111	rd	1110011	csr→rd, $\overline{UE32(uimm)}$& csr→csr（csr原子读、清零）

6、系统指令 ——环境调试/断点指令

- **环境调试/断点指令**：ecall和ebreak是陷阱指令，用于从用户程序陷入操作系统内核或者调试器，它们会在执行环境中产生一个精确的请求陷阱。
  - ecall指令：用于应用程序向操作系统内核发出系统调用的服务请求
  - ebreak指令：用于产生断点异常，从而将控制权交给调试环境
  - 执行环境接口EEI会定义如何传递服务请求的参数

系统功能调用

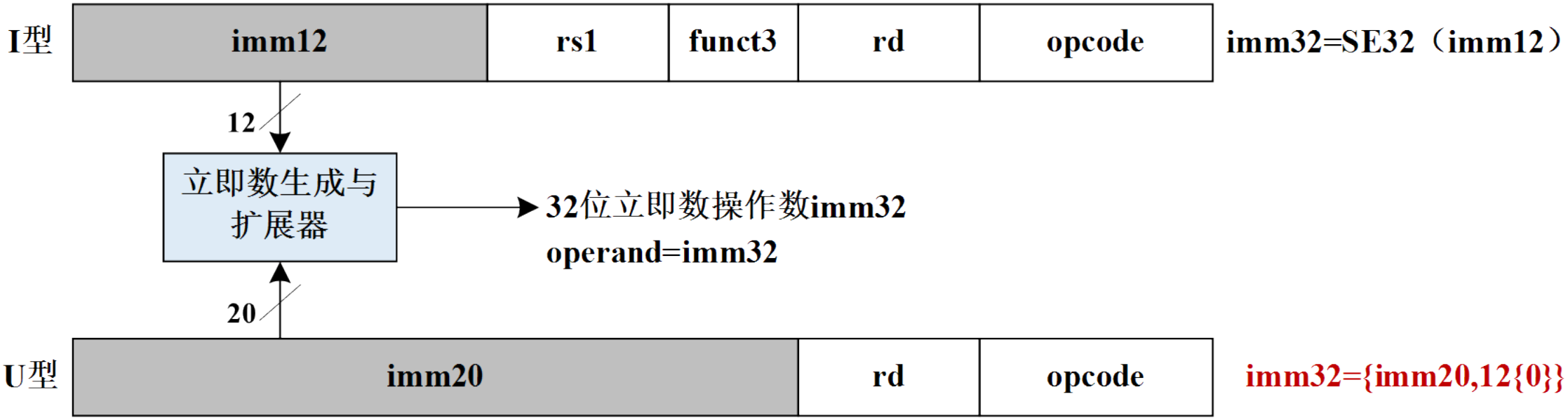
断点调试

I型调试/断点指令						
位数	12位	5位	3位	5位	7位	
字段	funct12	rs1	funct3	rd	opcode	指令功能
ecall	0000 0000 0000	00000	000	00000	1110011	
ebreak	0000 0000 0001	00000	000	00000	1110011	



6.6.5 RV32I的寻址方式

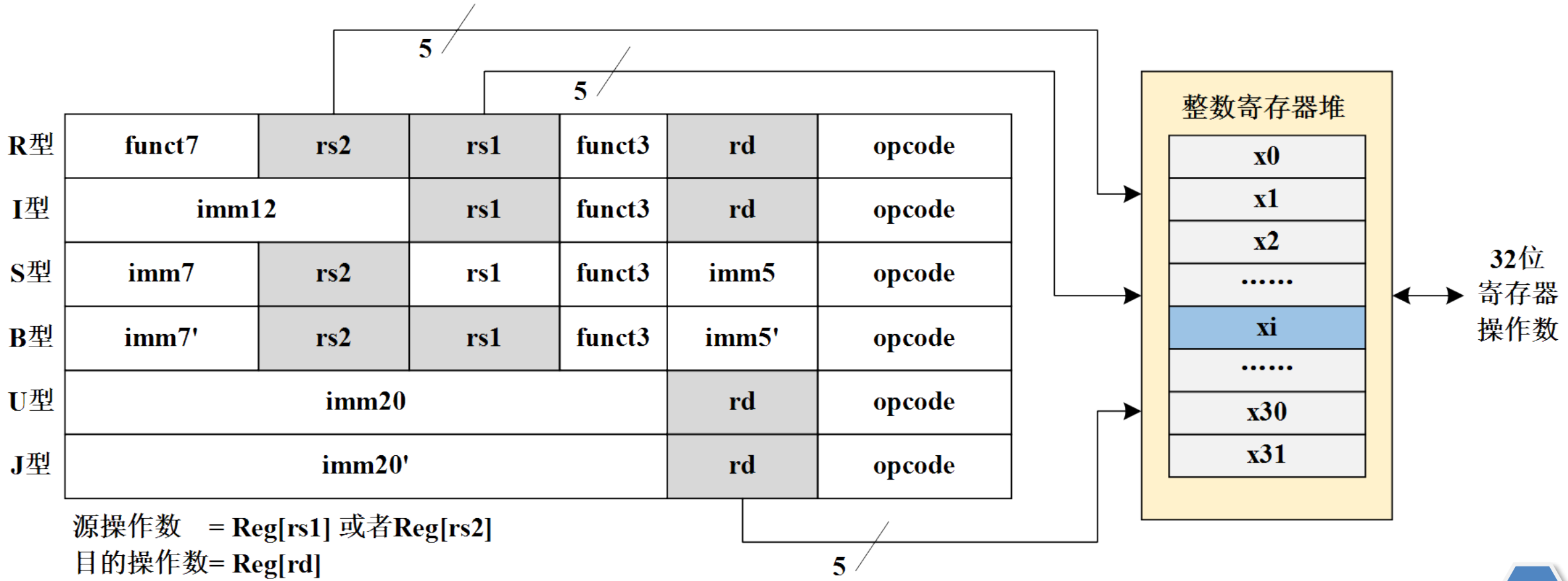
- RV32I指令集操作数：有立即数、寄存器和存储器三种类型
- 寻址方式有4种：
- 1、立即寻址
  - 操作数：由指令中的立即数字段指定，是一个常量，需要扩展为32位立即数



6.6.5 RV32I的寻址方式

❖ 2、寄存器寻址

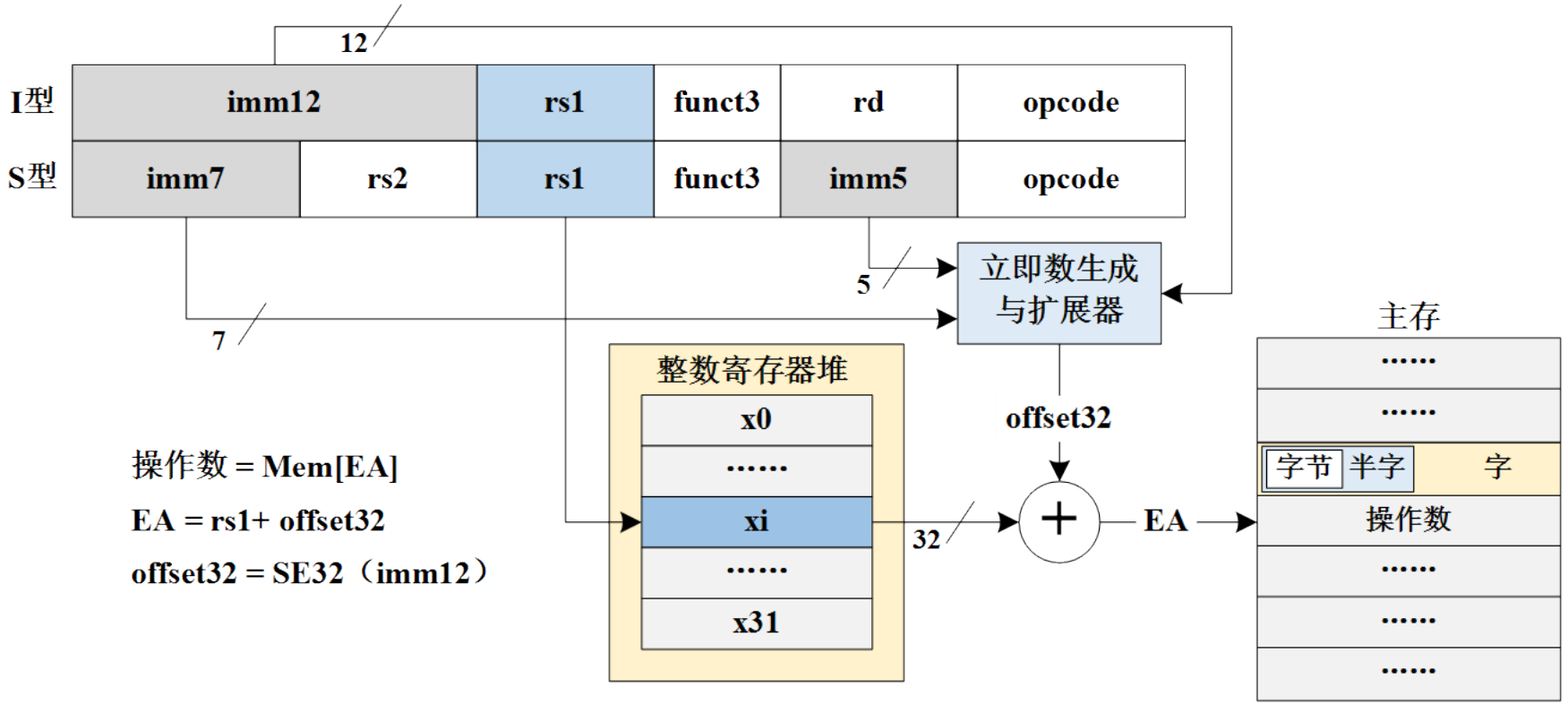
- 操作数：在某个整数寄存器中，由指令给出5位寄存器编号rs1、rs2、rd



6.6.5 RV32I的寻址方式

❖ 3、基址寻址：用于存数和取数指令

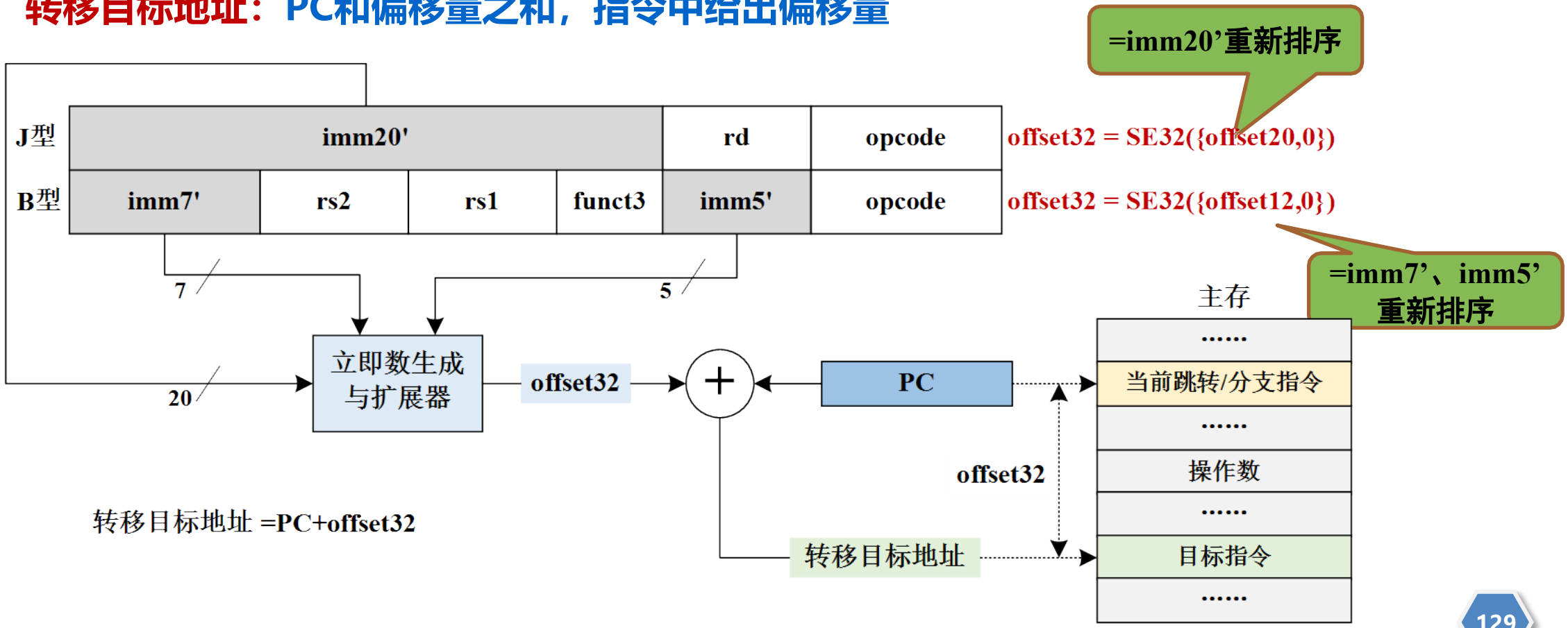
- 操作数：在主存的某个单元中；EA=rs1内容（base）+偏移量（offset）相加得到
- 指令：给出rs1的5位编号和12位偏移量（立即数字段）
- 偏移量：需要经过符号扩展变成32位偏移量，即EA = rs1+ SE32（imm12）



6.6.5 RV32I的寻址方式

❖ 4.相对寻址

- 主要用于J型和B型格式的无条件跳转、条件分支指令；
- 转移目标地址：PC和偏移量之和，指令中给出偏移量



6.6.5 RV32I的寻址方式

❖ 【例6.10】：假设PC=0000 0000h，内存部分内容如下表所示（地址和内容均以16进制表示），CPU启动执行一段RISC-V机器语言程序，请分析这段程序的每一条机器指令，以及程序的功能。

地址	内容	地址	内容
0	00500293h	1000 0200	11223344
4	10000337h	1000 0204	55667788
8	20032383h	1000 0208	99aabbcc
c	fff3c393h	1000 020c	ddddeeee
10	20732023h	1000 0210	0000ffff
14	00430313h	1000 0214	ffff0000
18	fff28293h	1000 0218	20212021
1c	fe0296e3h	1000 021c	99998888

6.6.5 RV32I的寻址方式

- 对机器指令的解码，即反汇编的过程；因为PC=0，故从0号单元开始执行程序。
- 第一条指令的解码过程：
  - ① 取出指令编码，写出二进制串：

    Mem[0]=0050 0293h = 0000 0000 0101 0000 0000 0010 1001 0011b

  - ① 提取操作码opcode，确定指令格式：opcode=0010011，是I型格式的运算指令
  - ② 按照格式分解各个字段，确定指令功能：

imm12	rs1	funct3	rd	opcode
0000 0000 0101	00000	000	00101	0010011

    - funct3=000，表明是addi指令；imm12字段是立即数5；rs1字段是0，表示x0寄存器，rd字段是5，表示目的寄存器为x5
    - 为：addi x5,x0,5指令，将立即数送入寄存器x5（即t0）
    - 可以简写为汇编的伪指令：li t0,5

6.6.5 RV32I的寻址方式

- 同理，解析后续的指令。对于B型和J型指令，可能需要调整立即数字段的编码顺序。程序的机器码解析结果：

地址	机器指令	汇编指令	汇编伪指令	功能	地址	执行后的内容
0	00500293h	addi t0,x0,5	li t0,5	5→t0	1000 0200	eeddccbb
4	10000337h	lui t1,10000h		10000 000h→t1	1000 0204	Aa998877
8	20032383h	L1: lw t2,200h(t1)		Mem32[t1+200h]→t2	1000 0208	66554433
c	fff3c393h	xori t2,t2,-1	not t2,t2	t2取反→t2	1000 020c	22221111
10	20732023h	sw t2,200h(t1)		t2→Mem32[t1+200h]	1000 0210	ffff0000
14	00430313h	addi t1,t1,4		t1+4→t1	1000 0214	ffff0000
18	fff28293h	addi t0,t0,-1		t0-1→t0	1000 0218	20212021
1c	fe0296e3h	bne t0,x0,-20	bnez t0,L1	if (t0≠0) PC-20→PC	1000 021c	99998888

- **程序功能：**对内存单元1000 0200h开始的连续5个字单元执行了与全1异或的操作（等价于取反：not），结果存回内存单元
- **执行结果：**从单元1000 0200h开始的前5个数据，每一位都被取反了

本章小结

- **概念：**
  - **指令系统：指一台计算机的所有机器指令的集合，又称为指令集**
  - **指令集体系结构ISA：处理器所支持的指令集（汇编指令集及机器指令编码）及其操作数的相关信息（寄存器、存储器、I/O等）**
  - **机器指令：能被计算机硬件识别并执行的特殊的0/1代码串（又称机器字），指定了计算机硬件的某种特定操作**
- **机器指令：由操作码字段和地址码字段组成**
  - **操作码字段：确定指令所要执行的操作**
  - **地址码字段：用来寻找执行指令所需要的操作数**
- **操作数类型：立即数类型、寄存器类型、存储器类型和I/O设备类型**
- **操作码扩展技术：将不使用的地址扩展为指令的操作码**
  - **缩短指令的平均长度，增加指令字所能表示的操作信息，但也增大了硬件设计难度**

本章小结

- **寻址方式：**
  - **指令寻址：确定下一条指令在主存中的地址，主要分为顺序和跳跃两种方式**
  - **数据寻址：用于获得本条指令执行所需要的操作数，有立即、直接、间接、寄存器、寄存器间接、变址、基址、相对、堆栈等9种基本寻址方式；将任意两种组合或者加以变化，可以实现复合寻址**
- **指令类型：大部分计算机的指令系统都包含数据传送、算术逻辑运算类、移位操作类、转移类、堆栈操作类、输入输出类等基本指令类型。**
- **指令系统的设计：应满足完备性、有效性、规整性、兼容性四个方面的要求**
- **CISC指令系统、RISC指令系统特点**
- **Intel x86、Yy-z02、RISC-V、ARM指令系统举例**
- **重点：寻址方式、操作码扩展技术、指令系统设计、机器指令的编解码（机器指令⇔汇编指令⇔高级语言）**





The End !